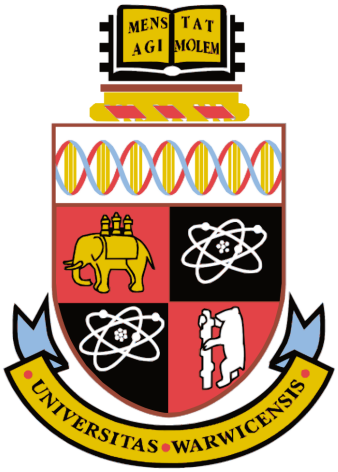


Competitive Analysis of Caching with Delayed-Hits on Line Networks

by

Saatvik Kambhampati
ID: 5714960

supervised by

Prof. Florin Ciucu

Submitted for the degree of Master of Science in

Computer Science

Department of Computer Science,
University of Warwick

October 17, 2026

Abstract

This report formalises a so-called 'delayed-hits-aware' line-network model, characterised by the number of caches $|V|$, the per-cache capacity k, and the inter-cache delay Z_A. It further establishes two lower-bounds on the competitive ratio of deterministic online placement policies. The former is a trivial lower-bound on all placement policies ($\Omega(1)$), which grows independently of $|V|$ k, and Z_A. The latter is a lower-bound specifically for the Leave-a-Copy-Everywhere (LCE) placement policy ($\Omega(k|V|)$). Then, finally, by trying to find a corresponding upper-bound, this report shows that LCE is unbounded above and is therefore an uncompetitive online algorithm.

This work was initially motivated by the fact that caching has become vital in the implementation of networked systems over long geographical distances. This is because it allows data to be stored closer to end-users, reducing overall retrieval latency and improving the quality of service of networked systems. A major example of this is with Content Delivery Networks (CDNs) that have been implemented globally.

The final contribution of this report is a proof that the LCE placement policy, is, in fact, an uncompetitive online algorithm. By providing a sequence that forces the LCE's latency to grow, not with network-specific parameters, but with *sequence*-specific parameters, it is shown that the LCE latency diverges from the optimal algorithm's latency, thereby proving uncompetitiveness. This lower-bound, and subsequent uncompetitive proof are two novel contributions to caching that work towards closing a major gap in the current literature. All results are then empirically validated by simulation work.

Contents

1 Introduction **1**
- 1.1 Outline & Contributions . . . 3
  - 1.1.1 Outline of Report . . . 3
  - 1.1.2 Contributions . . . 3

2 Problem & Objectives **5**
- 2.1 Problem Statement . . . 5
- 2.2 Objectives . . . 7

3 Background Research **8**
- 3.1 Classical Caching & Caching With Delayed Hits . . . 8
- 3.2 Eviction Policies & Content Placement Policies . . . 9
- 3.3 Competitive Analysis . . . 10
  - 3.3.1 Upper-Bound Analysis Techniques . . . 11
  - 3.3.2 Uncompetitive Policies . . . 11

4 Caching Model **13**
- 4.1 Formulation of Caching & Delayed-Hits . . . 13
  - 4.1.1 TTL Eviction Policy . . . 16
  - 4.1.2 Assumptions . . . 17
- 4.2 Cache Networks . . . 17
  - 4.2.1 The LCE Placement Policy . . . 19
  - 4.2.2 Cache Line Networks . . . 19
  - 4.2.3 Assumptions for Cache (Line) Networks . . . 21
- 4.3 Competitive Analysis for Caching . . . 21
- 4.4 Finding the Lower-Bound . . . 23
- 4.5 Finding the Upper-Bound . . . 23

5 Competitive Analysis of LCE on Line Networks **25**
- 5.1 Choosing $\mathcal{O}$ For CPPs . . . 25
- 5.2 Initial Lower-Bound . . . 26
- 5.3 Lower-Bounding LCE . . . 31

5.3.1 Adversarial Sequence for LCE . . . 31
5.3.2 LCE Lower-Bound . . . 33
5.4 LCE has no Upper-Bound . . . 36
5.4.1 A Family of Sequences That Violate Theorem 4.3.2 . . . 37

6 Simulation Results **39**
6.1 Framework Design . . . 39
6.1.1 Structures, Objects & Classes . . . 40
6.1.2 Generalised Algorithms . . . 40
6.1.3 Implementation of $\mathcal{O}$ . . . 42
6.1.4 Dynamic Creation of Sequences . . . 42
6.2 Simulation Specifics . . . 43
6.2.1 Simulation Flowchart . . . 44
6.3 Simulation of ρ_{MW} on Line Networks . . . 45
6.4 Simulation of ρ_A on Line Networks . . . 46
6.5 Simulation of $\{\rho_n\}$ . . . 47
6.6 Further Verification of Theory Using Cumulative Latency Graphs . . . 48

7 Exploration of Tree Topologies **52**
7.1 Formal Model . . . 52
7.2 Multiple Requests . . . 53
7.3 Considerations for $\mathcal{O}$ . . . 53
7.4 LCE may also be Uncompetitive on Trees . . . 54

8 Conclusion and Evaluation **55**
8.1 Project Management . . . 55
8.1.1 Unforeseen Problems . . . 55
8.1.2 Appraisal and Reflection . . . 56
8.1.3 AI Usage . . . 56
8.2 Conclusion . . . 57
8.3 Evaluation . . . 57
8.4 Future Work . . . 58
8.4.1 Formal Definition for $\mathcal{O}$ . . . 58
8.4.2 Generalisation to Other or All Topologies . . . 59
8.4.3 Extension to Other CPPs, or Exploration into CPP-Invariant Bounds . . . 59

A Timetable **60**
A.1 Overall Initial Timetable . . . 60
A.2 Post-Interim Timetable . . . 64

List of Definitions, Theorems, and Lemmas

4.1.1 *Definition* Sequence of Requests . . . 13
4.1.2 *Definition* Cache and Cache State . . . 14
4.1.3 *Definition* Hits and Misses . . . 15
4.1.4 *Definition* Fetch Times . . . 15
4.1.5 *Definition* Latency . . . 16
4.1.1 *Assumption* Single R . . . 17
4.1.2 *Assumption* Discrete times T . . . 17
4.2.1 *Definition* Cache Networks . . . 18
4.2.2 *Definition* Inter-Cache Delay Parameter . . . 18
4.2.3 *Definition* Non-Zero-Latency-Hits . . . 19
4.2.4 *Definition* Cache Line Networks . . . 20
4.2.1 *Assumption* Homogenous Z_A . . . 21
4.2.2 *Assumption* Homogenous k . . . 21
4.3.1 *Definition* Competitive Ratio (Closed Form) . . . 21
4.3.2 *Definition* Competitive Ratio . . . 22
4.3.1 *Theorem* Competitive Ratio Lower-Bound . . . 22
4.3.2 *Theorem* Competitive Ratio Upper-Bound . . . 22
4.3.3 *Theorem* Manohar & Williams's Lower-Bound . . . 23
5.2.1 *Definition* Bursty Request . . . 27
5.2.2 *Definition* Pure Request . . . 27
5.2.1 *Lemma* $\mathcal{P}$'s Latency . . . 28
5.2.2 *Lemma* $\mathcal{O}$ Hits on all Bursties . . . 29
5.2.3 *Lemma* $\mathcal{O}$'s Latency . . . 29
5.2.1 *Theorem* Initial Lower-Bound . . . 30
5.3.1 *Definition* The Set W . . . 31
5.3.1 *Example* The (2,2) Network . . . 32
5.3.2 *Definition* Adversarial Sequence ρ_A . . . 32
5.3.1 *Lemma* LCE's Latency . . . 33
5.3.2 *Lemma* $\mathcal{O}$ Hits on all Bursties . . . 34
5.3.3 *Lemma* $\mathcal{O}$'s Latency . . . 34
5.3.1 *Theorem* LCE Lower-Bound . . . 35
5.4.1 *Corollary* Uncompetitiveness . . . 36
5.4.1 *Theorem* LCE is Not Competitive . . . 38
7.1.1 *Definition* Cache Tree Networks . . . 52

List of Figures

2.1 A 'single-cache system'. . . . 6

4.1 The classical caching pipeline. 14
4.2 A line network of $|V|$ caches 20

6.1 UML Class Diagram for Main Classes 40
6.2 A flowchart showing the process for each simulation herein. 44
6.3 ρ_{MW} comparison for different fixed parameters 46
6.4 ρ_{A} comparison for different fixed parameters 47
6.5 Enter Caption 48
6.6 Cumulative Latency of any $(k, |V|, Z_A)$-network configuration using ρ_{MW} 49
6.7 Cumulative Latency of any $(k, |V|, Z_A)$-network configuration using ρ_A 49
6.8 Cumulative Latency of any $(N_\mu = 32\ 768, k = 8, |V| = 8, Z_A = 16)$-network configuration using $\{\rho_n\}$ 50

7.1 A tree network of caches 53

A.1 Gantt Chart March - April 61
A.2 Gantt Chart June - August 62
A.3 Gantt Chart August - September 63

Chapter 1

Introduction

Suppose a user in India wishes to access a company's website whose server is located in the United States. Traditionally, the user would have to wait for their request to travel between both locations, twice, which could take a relatively long amount of time. This, therefore, provides a low 'quality of service' to the end-user. Now suppose instead that the same company owns servers across the planet, and has one in India. These distributed servers are then used to store copies of the same website, and they might also hold the data for other websites. Now, the user in India need only make a request to the server located in India, instead of all the way to the US. This drastically reduces the time taken for the website to load on the users' computer, and improves the quality of service significantly. This technique is know as a 'Content Delivery Network' (CDN). CDNs, essentially, act as 'middlemen' for serving content [1] and provide low-latency access to frequently-requested data that may otherwise take far longer for a user to retrieve [2].

CDNs belong to a broader technology known as 'caching', which has traditionally been deployed across a wide range of applications, from CPU hierarchies to networked systems, as stated above. In recent years, the importance of caching, as a technique to improve and contribute to the quality of service of systems, has become increasingly recognised [3].

To explain the caching process, consider again the example above. The server (the 'cache') located in India only has a finite amount of storage, and users may be requesting more websites from it than it can physically store. Thus, when a user makes a request the cache checks if it has the website currently stored. If it does, it serves the website (this is known as a *cache-hit*), if it doesn't (this is known as a *cache-miss*) [2], then the request is automatically forwarded to the original server in the United States (this is called a *fetch request*). Then, upon serving this fetch request, the cache can choose whether or not to store this website in the cache. However, if the cache is full, then it must first 'evict' a website that is already stored.

Any request can impose a latency on the user, depending on how long the request takes to be served. Intuitively, requests that are served by a geographically closer

cache incurs less latency than a further one. Furthermore, requesting an item from the original server (traditionally known as the *backing-store*), in any non-trivial scenario, incurs the most amount of latency. An extension of this latency is the *delay-parameter*, introduced by Atre et al. [4]. This is the ratio of the fetch time to the inter-request arrival time[1]. This property is essentially a statement about the number of requests that arrive at the cache while a fetch request for that same item is already en-route.

Thus, due to latency, caching becomes a balance of trade-offs: Do you evict a website that you have already got in favour of one that has just been requested? What if the website you just evicted gets requested later, but the newly-cached one is never requested again? In caching, there exists algorithms that must decide whether or not the 'eviction candidate' is more popular than the 'admission candidate'. They must also, in some way, decide on an eviction candidate out of all the items currently stored. Such algorithms that make these decisions are called *policies*. Traditionally, there exists three types of policies:

- The 'admission' policy, which determines whether a fetched item is to be admitted into the cache;
- The 'eviction' policy (sometimes called 'cache replacement policy'), which decides what item already in the cache should be evicted [5]; and
- The 'content placement policy', which determines at *which* cache in the network an item will be admitted to.

In 'classical caching', delay parameter is set to zero (implying that the fetch time is zero). However, real systems may have a tangible, non-negligible lag between the cache and the backing store, such that the fetch time is *not* zero, as in the example provided above. This has been observed in high-throughput systems, such as in FPGAs [4], and also in CDN hierarchies [2]. Suppose that two requests for the same item are to arrive at a cache, which does not have this item. The first request results in a cache-miss, and a fetch request is sent to the backing-store. Then, the second request arrives while this response is still in-flight. In a classical model, this results in a cache-hit as the item is instantly returned from the backing-store and admitted to the cache. However, in non-classical models, the request is placed into a queue. When the fetch for the first request is resolved, the second request is taken out of a queue and is considered a *delayed-hit* (DH). Models in which DHs are accounted for are identified as 'DH-aware' contexts, with the converse being 'DH-unaware' contexts.

The behaviour and process for caching works slightly differently in *cache networks* (such as CDNs), compared to so-called 'single-cache systems'. A cache network is, informally, a group of inter-connected caches that can forward requests to one-another (if they cannot serve the request themselves). The network, ultimately, is connected to one backing-store, and may have one or more 'ingress' caches (caches to which requests

[1]The inter-request arrival time is the time between two requests for the same item.

are initially sent). Such a network allows for more data to be stored over a larger physical distance. This means that it can serve geographically dispersed clients with lower latency. Furthermore, a cache serving a client in one area may store data that is relevant to that area specifically, and may query more distant caches for data that is less frequently requested.

1.1 Outline & Contributions

1.1.1 Outline of Report

This report first starts with Chapter 2, which outlines the specific problem statement that is being addressed, and the motivation behind it. It further provides the objectives that guide this project, and against which its success will be measured. Then, Chapter 3 covers an extensive literature review of caching research, most particularly of competitive analysis and online algorithms. The caching models used in this project are then formally defined in Chapter 4, along with the specific policies/algorithms used. The same section also explains, in-depth, the concept of competitive analysis. In Chapter 5, the central contributions of this project are outlined, along with the accompanying proofs to relevant statements. Chapter 6 introduces, firstly, the simulation framework used in this project. Then, it uses the provided framework to validate the theoretical results obtained in Chapter 5. Chapter 7, finally, explores tree topology cache networks, and how a different network topology may affect certain behaviours in the policies, or how it constrains their actions.

1.1.2 Contributions

The central contribution of this project is proposed in Chapter 5. This is a theorem showing that the Leave-a-Copy-Everywhere placement policy is not competitive in a line-network of caches with delayed hits. This is shown by iteratively providing various candidates for the competitive ratio lower-bound, and, when attempting to find a matching upper-bound, instead concluding that the ratio is unbounded in this specific model.

First, the current algorithm to construct an adversarial sequence that yields a lower-bound is reviewed. This algorithm was specifically designed for single-cache systems. Then, this is applied to cache-networks, providing a novel lower-bound value that acts as a guide for further investigation. Then, by iterating on this approach, a novel sequence is constructed specifically for cache-networks that provides a tighter lower-bound. Finally, whilst attempting to find a matching upper-bound to prove tightness, the chosen policy is proven to be *uncompetitive*. That is, a theorem is provided that proves it is unbounded above, which is the major contribution of this project.

The second, auxiliary contribution is the simulation framework in Chapter 6. This

is used to validate the theoretical results obtained in Chapter 5. The chapter outlines how the framework was developed, the specific parameters used in the simulation, and the results of each simulation.

In this chapter, an informal introduction to caching, delayed-hits, and cache networks have been outlined, along with an explanation of the motivation behind exploring this problem. The next section expands on this by providing a more thorough explanation of the motivation, justifying the provided problem statement. It will then outline the core objectives of the project, based on the provided problem statement.

Chapter 2

Problem & Objectives

In this chapter, the problem statement and the derived objectives of the project are more thoroughly discussed. Specifically, a brief overview of relevant policies or concepts in caching are explored within cache networks to build the intuition behind the problem that is being explored and the research gap being addressed.

2.1 Problem Statement

Much of the current caching literature only analyses single-cache systems with delayed-hits *or* caching networks under non-DH conditions. There is little literature regarding competitive analyses of cache networks *with* delayed-hits, which is the gap that this project aims to contribute to. It aims to extend the competitive analysis of DH-aware caching to the line-network topology, from single-cache systems. Specifically, the behaviour of the Leave-a-Copy-Everywhere (LCE) content placement policy (CPP)[1] considered on a line network in a DH-aware context. In LCE, when the backing-store returns a fetch, the item is admitted to every cache along the queried path. In this way, LCE introduces a high amount of redundancy [6]. Despite this, LCE specifically is studied here as it is heavily applied in literature, as a baseline policy [6, 7], and in industry [8]. Thus, it is the best policy on which to perform an initial competitive analysis. The central contribution of researching this specific gap is to provide a tight competitive analysis of LCE on DH-aware caching networks.

In DH-unaware contexts, a cache hit incurs zero-cost, regardless of from where the item is fetched [4]. DH-aware contexts fundamentally change this. If a cache fires a miss and forwards a fetch request, and the next cache in the network fires a hit, the delay incurred is the latency between two adjacent caches. This is less than the latency of forwarding to a deeper cache, or to the backing-store. Thus, hit cost is now proportional to the 'fetch distance'. This provides an interesting problem to be solved

[1] A CPP is a policy that determines *to which cache* a specific item should be admitted, in contrast to an eviction policy which determines *what* should be evicted from a specific cache

in the context of competitive analysis for LCE in a network, as hits are not simply latency-free anymore and so must also be considered in counting the latency.

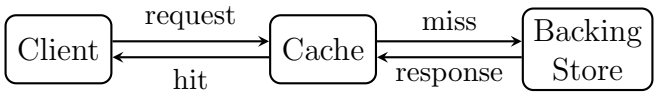


Figure 2.1: A 'single-cache system'.

2.2 Objectives

The objectives for this project are as follows:

1. Formally define the DH-aware line caching model, including the cost function and the LCE placement policy;
2. Construct an adversarial request sequence establishing a lower-bound;
   (a) Establish a baseline lower-bound using Manohar & Williams's adversarial sequence [9], which will help guide future work;
   (b) Establish the lower-bound for LCE;
3. Prove an upper bound on the competitive ratio (CR)[2] of LCE on a line of caches as a function of the number of caches, size of each one, and the delay parameter, thereby showing tightness;
4. Develop a DH-aware cache network simulator capable of modelling line topologies under configurable policies, cache sizes, and delay parameters;
5. Validate the theoretical bounds experimentally by comparing simulated CRs against the proven guarantees; and
6. Investigate generalisations to tree topologies as extensions.

[2]The CR is a metric to evaluate the performance of algorithms in comparison to their optimum counterparts, and is introduced in more detail in Chapter 4.3

Chapter 3

Background Research

The last Chapter discussed the problem statement in more detail, and provided the derived objectives of this project. The next step, then, is to conduct a more in-depth review on the current landscape of caching & competitive analysis literature, which is provided herein. The review focuses on the beginnings of caching, and the development of delayed-hits. It also covers the publication of the first offline optimal algorithm, before moving on to a short review of relevant eviction and placement policies. For competitive analysis, the review begins with the foundational paper in the field, and then discusses more recent work upon which this project is based. It finishes with a review of upper-bounding techniques, and the edge-case of so-called 'uncompetitive' algorithms.

3.1 Classical Caching & Caching With Delayed Hits

The concept of caching can trace its origins, via 'paging', back to Kilburn et al.'s seminal 'One-Level Storage System' paper, published in 1962 [10]. This system discussed a novel memory storage technique that was used in the Atlas computer. They originally proposed the idea of the 'core-drum store' system. What would appear to the user as a single level of storage would, internally, act as two components: a 'core-store' that could query a 'drum-store' for additional data. This technique eventually became known as 'paging'.

Following this, in 1966, L.A. Belády proposed his optimal, offline eviction policy Min [11]. In his paper, he states that no realistic (that is, online) eviction policy can be the 'optimal' policy because they are not able to take into account future requests. Thus, the most optimal policy should be based on such information being available. That is, the optimal policy must be an *offline* one. Such policies should read the entire input sequence, and then work backwards to build the 'minimum replacement sequence' [11]. The Min algorithm itself is fairly straight forward: Evict the item which will not be requested for the *longest* time into the future. This has been the standard

optimum, offline algorithm for evictions used in *classical* caching, since its publication.

Then, in 2020, Atre et al. [4] highlighted the effect that DHs could have on caching systems. They show that, by assuming the existence of a non-negligible fetch time, a request for an item that arrives during an existing fetch request for that item should not be classified as a cache-miss or a cache-hit. Atre et al. argues that neither is 'correct'. Instead, they propose a third classification: the delayed-hit. They argue that the cost incurred by this second request is in fact less than that of the first (because the second request 'waits' less time for the first request to be fulfilled). Thus, they classify it as a *delayed-hit*. They show the sub-optimality of Belády's MIN [11], when observed in a DH-aware context. Furthermore, they discuss the discrepancy between hit ratio and latency, whilst providing an alternative DH-aware metric: the Aggregated Delay. They then propose a new latency-optimal offline algorithm known as BELATEDLY. Finally, they introduce a novel online algorithm for DH-aware caching known as MINIMUM-AGGREGATEDELAY.

3.2 Eviction Policies & Content Placement Policies

Since the introduction of caching and paging, certain policies and algorithms have been designed that act as decision rules, to determine whether specific items are to be evicted or admitted to the cache. Further, in cache networks, there exist content placement policies that dictate *at which cache* an item will be placed in the network. All three of these have been extensively studied, however eviction policies remain the focus of much caching literature.

Popular eviction policies include LEAST-RECENTLY-USED (LRU) and LEAST-FREQ-USED (LFU), both of which have been combined to form the LEASTRECENTLY-FREQUENTLYUSED (LRFU) policy by Li et al. in 2012 [12]. In all three policies, a number is associated with every item in the cache. In LRU, this number represents the time at which the item was last requested (and, upon eviction, the item with the time furthest in the past is evicted). Whereas, in LFU, this number is a counter of how many times the item has been requested since it was last admitted to the cache (and the item with the lowest frequency is evicted). Li et al. then propose the LRFU policy for caching routers in 2012, combining the advantages of both policies. They do this by introducing a **C**ombined **R**ecency and **F**requency value (CRF), which is computed by using a function $f(x) = e^{-\gamma x}$, where $\gamma \in [0, 1]$ represents a weighting that allocates greater importance to either the *recency* or *frequency* property. They provide the computation of the CRF value for an object i at time t_i as

$$C_{t_i} = \mathrm{CRF}(t_i) = \begin{cases} f(0) + f(t_i - t_i')C_{t_i'} & , a(i) = 1 \\ f(0) & , a(i) = 0 \end{cases}$$

The TIME-TO-LIVE (TTL) eviction policy is also a staple of caching. TTL works

by, canonically, evicting any items whose associated timer (counting down from their time-to-live value) reaches zero. They have further been noted for their relationship with and mapping to capacity-driven policies, such as LRU [13].

Separately, placement strategies for caching networks have been studied in DH-unaware contexts. Strategies such as LCE and Leave-a-Copy-Down (LCD) move content toward the client on cache hits [7]. An alternative approach, Demote [8, 14], instead pushes evicted items one hop upstream in the cache hierarchy on replacement, preserving recently-evicted content deeper in the network. While LCE is very prevalent in the literature, it has not been analysed under the DHs model or subjected to competitive analysis.

3.3 Competitive Analysis

Competitive Analysis is a method, first pioneered by Sleator & Tarjan in 1985, that attempts to describe the worst-case performance of an *on-line* algorithm relative to the optimal, *off-line* algorithm. It involves, centrally, the computation of the *competitive ratio*, which is the ratio of the 'cost' incurred by the online algorithm to the cost incurred by the optimal, offline algorithm, given an input sequence. Slator and Tarjan proved that the LRU eviction policy is $O(k)$-competitive (where k is the cache capacity). The optimal offline policy that they used was Belády's Min algorithm [11]. The competitive ratio upper- and lower-bounds were later defined again by Karlin et al. [15] in 1988, who originally coined the term *Competitive Analysis*.

In 2020, Manohar and Williams [9] published a paper that proposed a competitive ratio lower-bound for any deterministic DH-aware online caching algorithm. Specifically, they show that the lower-bound grows with both the capacity of the cache and the delay parameter. In this paper, they further prove that Atre et al.'s DH model does not exhibit anti-monotonicity and provide the required modifications to rectify this. Anti-monotonicity is a valuable property to retain in caching as, intuitively, a cache hit should result in a lower latency than a cache miss. Their modification is to simply fetch an item from the backing store, *even* when the request results in a cache-hit, resulting in a model that never 'rewards' a miss more than a hit.

In 2025, Gurushankar et al. [16] further prove that the Least-Recently-Used (LRU) eviction policy, in a DH-aware context, has a CR upper-bound that scales with the cache capacity and *also* with the delay parameter. The two bounds 'match', meaning that this provides the first 'tight guarantee' for delayed-hits caching with the LRU. They prove that this guarantee extended to any marking algorithm. That year, Tan et al. [17] published a paper that shows any randomised caching algorithm has a CR lower-bound that scales with the cache capacity and the logarithm of the delay parameter ($\Omega(k \log Z)$). They then provide the outline for a novel randomised caching algorithm 'CaLa', proving the upper-bound to be $O(k \log Z)$-competitive, thereby matching the bounds and providing a tight guarantee.

3.3.1 Upper-Bound Analysis Techniques

Upper-bound analysis can be achieved through a variety of methods and techniques. Initially, Sleator & Tarjan introduced the phase-partitioning technique, designed specifically for marking algorithms [18]. In such a technique, the input sequence can be partitioned into distinct phases, determined by the capacity of the cache. Then, by making an argument about how many misses there are at most or at least per-phase, the upper-bound of the CR can be determined. This was then extended more formally in 1991 by Fiat et al., who showed that randomised online algorithms were H_k-competitive via phase partitioning [19]. Gurushankar et al.'s novel introduction of superphase partitioning stems directly from this [16].

Buchbinder and Naor's 2009 publication [20] introduced the idea of using Primal-Dual linear programs for competitive analysis, and its capabilities in aiding the design and analysis of online algorithms. They show that an online algorithm for a problem can be derived by reinterpreting the fact that the Dual LP acts as a lower-bound for the Primal LP. This, in turn, can aid in identifying an upper-bound. Initially, they use the Primal-Dual approach to solve the *weighted paging problem*, in which every page (item) has a fetch-cost associated with it, that is charged on cache-misses. By redefining this cost on *evictions* instead (which they then 'bind' to misses), they provide a Primal-Dual program to eventually provide an $O(\log k)$-competitive 'fractional' algorithm. Then, Bansal et al. further show that a randomised algorithm in the weighted paging problem was also $O(\log k)$-competitive via the Primal-Dual approach.

At around the same time, Bansal et al. (*) provided an $O(\log^2 k)$-competitive algorithm for the *generalised caching problem* – which is where each item in the universe has its own arbitrary weight/cost. Adamaszek et al. would later improve on this by introducing a new $O(\log k)$-competitive algorithm for generalised caching [21]. Both papers relied upon the Primal-Dual approach for this problem. They further reported that this improved bound unifies all edge-cases of the generalised caching problem, and matched the lower-bound, thereby providing tightness.

3.3.2 Uncompetitive Policies

As explained later, in Chapter 4.3, the CR upper-bound must hold across the whole sequence space. However, eviction policies can sometimes be *unbounded*: There exists no upper-bound that holds for the whole sequence space, in-terms of the specific parameters or variables of interest [22]. Notable examples of this are the LFU and Last-In-First-Out (LIFO) policy [22]. In LIFO, the cache acts as a stack in that the *last* item that was added is the item that is evicted [22]. Both of these policies can be proven to be unbounded. Such policies may also be referred to as *not competitive*. Borodin & El-Yaniv establish the uncompetitiveness of both [23]. This is explored further in Chapter 5.4.

Outside of caching, Krumke [24] attempted to use competitive analysis for the Dial-

A-RIDE Problem (DARP) in which users may request a 'ride' on-demand, as needed. Krumke attempted to bound the 'flow-time' of a request for a ride, which he defined as 'the difference between its completion time and its release time' – which is identical to the *latency* of a request in caching systems. He proves that his REPLAN algorithm for DARP scheduling is unbounded (and, thus, uncompetitive) because there is a specific instance in which REPLAN's competitive ratio grows to infinity, independent of the desired variables.

Chapter 4

Caching Model

In this chapter, all relevant caching models will be formally defined and introduced, along with notation regarding competitive analysis. First, the chapter defines the key aspects of a single-cache system before building upon this by introducing delayed-hits and then general cache networks. Once cache networks are introduced generally, the chapter then specifically defines line networks as a subset of cache networks by providing the criteria required for any cache network to be a line network. Herein the specific eviction policy (TTL) and placement policy (LCE) being used for the competitive analysis in Chapter 5 are also formally introduced.

4.1 Formulation of Caching & Delayed-Hits

Below, each part of the caching pipeline and model in a classical caching context is defined. First, a sequence of requests is defined, along with a cache and its specific state. This pipeline is then shown visually in Figure 4.1.

Definition 4.1.1 (Sequence of Requests): Given a set of all items $\mathcal{I}$ and a set of time $T \subset \mathbb{R}_{\geq 0}$, let a request r_j be modelled as a tuple s.th. $r_j = (i, t)$ for $i \in \mathcal{I}, t \in T$. Then

$$\rho = (r_j)_{j=1}^{N}$$

is a sequence of requests.

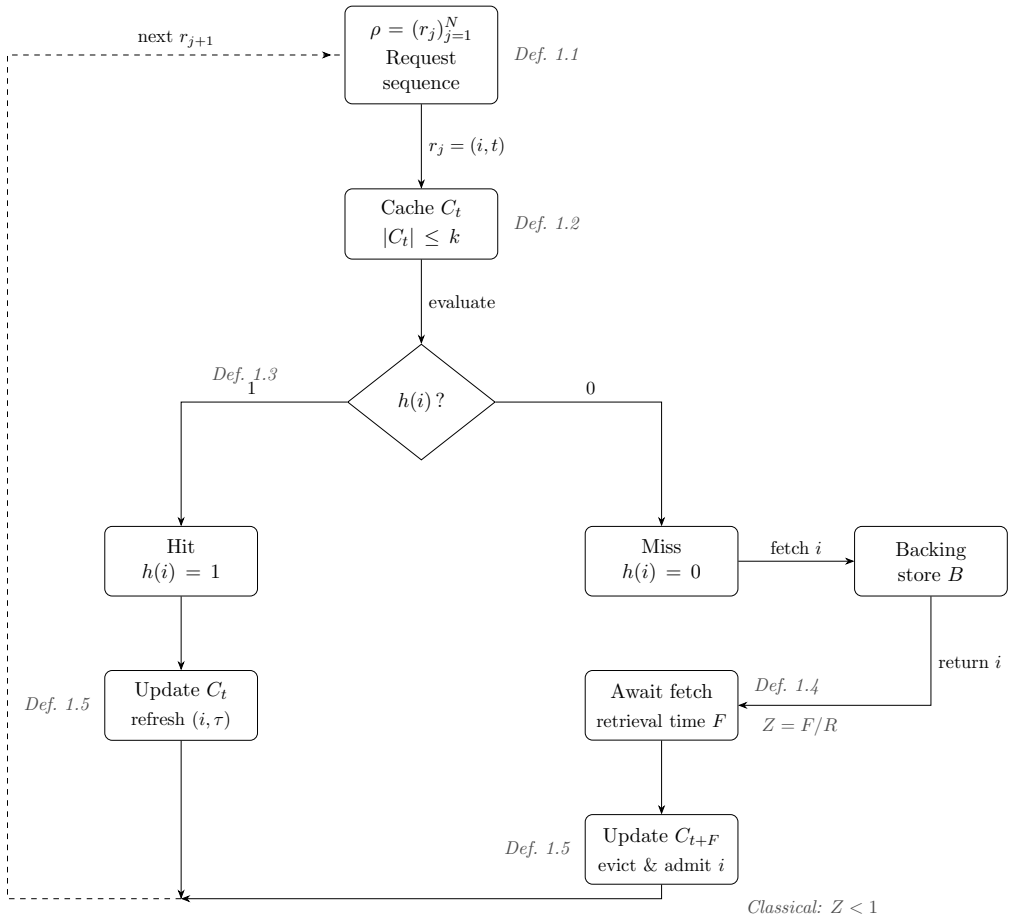


Figure 4.1: The classical caching pipeline.

Definition 4.1.2 (Cache and Cache State): Given a set of all items $\mathcal{I}$ and times $T \subset \mathbb{R}_{\geq 0}$, the state of a cache at a given time t is defined as a set

$$C_t = \{(i, \tau) \mid i \in \mathcal{I}, \tau \in T\} \subset \mathcal{I} \times T$$

Where τ represents some arbitrary and context-dependent time value that is associated with item i in the cache at state C_t. For example, in Time-to-Live policy caching, τ may represent the time-to-live of $(i, \tau) \in C_t$.
Hereinafter, a cache in general may be referred to as simply C, rather than its specific state at time t (C_t).

The sequence of requests represent the items that will be requested from the cache. Each request is sent in the order of the sequence, and at the times determined by t. Then, when the request arrives at the cache, the cache checks if it has the item being requested. This process then triggers a *hit* or a *miss* signal:

Definition 4.1.3 (Hits and Misses): Given a time $t = t_1 \in T$, the request at t is $r_j = (i, t_1)$ and the cache state is C_{t_1}. Then, a *hit indicator* function $h : \mathcal{I} \to \{1, 0\}$ is defined as:

$$h(i) = \begin{cases} 1, i \in \{i' \mid (i', \tau) \in C_t\} \\ 0, i \notin \{i' \mid (i', \tau) \in C_t\} \end{cases}$$

, where $h(i) = 1$ means *hit*, and $h(i) = 0$ means *miss*.

Then, given the above process, the total *fetch time* for an item (as explained informally in Chapter 1) is defined as follows. Subsequently, the expression for the delay-parameter is also defined:

Definition 4.1.4 (Fetch Times): At some time $t = t_1$, given the arrival function $h(i)$ in Definition 4.1.3 and a backing store $B \subset \mathcal{I}$, if $h(i) = 0$ (i.e. a miss is fired), C triggers a *fetch* request of item i from B. B then returns i to C at time $t = t_b$. Then, the *retrieval time* of this operation is $F = t_b - t_1$.
By letting the *inter-request arrival time* be R, the time to fetch can be expressed as the ratio of retrieval time to inter-request arrival time:

$$Z = \frac{F}{R}$$

[4]

After a hit or miss is triggered by the cache, the specific caching policy is called to update the cache state. This is shown in Algorithm 1. Note that $\mathcal{P}$ here represents the eviction policy, an example of which is defined in Algorithm 2 (TTL). $\mathcal{P}$ returns the item that should be evicted from the cache.

The algorithm firstly checks whether the hit-indicator has returned a hit or a miss for the current request. If it is a miss, then firstly wait Z time-steps for the item to return (during this time, the cache can accept and process other incoming requests). Then, call policy $\mathcal{P}$ to find the eviction candidate. Finally, use the following set-wise update rule:

$$C_{t'} \leftarrow \underbrace{(C_t \setminus \{(i_{\text{evict}}, \tau_{\text{evict}})\})}_{\text{evict old item}} \cup \underbrace{\{(i, \tau)\}}_{\text{insert new item}}$$

If the hit-indicator returns a cache-hit, however, then the τ' value that is associated with item i in the cache is simply updated to some new value. This value, as stated previously, is policy-dependent. In TTL, for example, it is a live counter of how many more time-steps until the time has 'expired', where as in LFU, it is a counter of how many times the item has been requested. The update is done by removing the pair

Algorithm 1: UPDATE

Data: Cache state C_t, known as C, an incoming request $r_j = (i, t)$, and an eviction candidate $(i_{\text{evict}}, \tau_{\text{evict}})$.
Result: $C_{t'}$
if $h(i) = 0$ **then**
  Wait Z time-steps for B to return item i.
  $(i_{\text{evict}}, \tau_{\text{evict}}) \leftarrow \mathcal{P}(C_t)$ `/* Use the eviction policy to find a victim to evict. */`
  $C_{t'} \leftarrow (C_t \setminus \{(i_{\text{evict}}, \tau_{\text{evict}})\}) \cup \{(i, \tau)\}$ `/*` $t' = t + Z$ `here. */`
end
else if $h(i) = 1$ **then**
  $C_{t'} \leftarrow (C_t \setminus \{(i, \tau')\}) \cup \{(i, \tau)\}$ `/*` $t' = t + 1$ `here. Since the item hit, no need to evict anything. Simply update or reset` τ, `and only if needed. */`
end
return $C_{t'}$

(i, τ') from the cache, and then immediately readmitting it with the reset τ-value:

$$C_{t'} \leftarrow (C_t \setminus \{(i, \tau')\}) \cup \{(i, \tau)\}$$

In classical ('DH-unaware') caching, it is assumed that $Z \leq 1$, which means that the retrieval time is assumed to be essentially instantaneous. However, in DH-aware caching models, $Z > 1$.

The advent of DH-aware models means that the time between a request arriving and being fulfilled (the *latency*) becomes a tangible, meaningful metric through which a caching system can be measured. Manohar & Williams defined latency as follows:

> ***Definition 4.1.5 (Latency):*** Given a single-cache system using a policy $\mathcal{P}$ and a sequence of requests $\rho = (r_1, r_2, r_3, \ldots, r_N)$, the latency of each request l_t is defined as:
>
> $$l_t = (1 - h(i))(Z - (t - p_t)),$$
>
> where p_t is the last request for the same item i requested at time t that resulted in a *cache-miss*.

4.1.1 TTL Eviction Policy

Time-**T**o-**L**ive is a specific type of eviction policy in which every item in the cache has a counter (the 'TTL'). At every time-step, this counter counts down to zero. Two variants of TTL are standard in caching. In the first, the item is evicted *as soon as* the TTL is zero. In the second, items are only evicted when a new item must be admitted

to the cache – at which point the policy chooses to evict the item with the lowest TTL. In this project, the latter variant was used. The analysis provided herein holds for all TTL values.

Algorithm 2: TIME-TO-LIVE

Data: The cache state at time $t + Z$, C_{t+Z}, known as C.
Result: The tuple $(i_{\text{evict}}, \tau_{\text{evict}}) \in C_{t+Z}$ that should be evicted.

```
victim ← (⊥, ⊥) ;      /* The (current) eviction candidate / victim */
for p ∈ C do
    ;                                   /* p is of the form p = (i_p, τ_p) */
    if τ_p < τ_victim ∨ τ_p = ⊥ then
        victim ← p
    end
end
return p
```

4.1.2 Assumptions

This section outlines some assumptions that are consistently used throughout this project.

Assumption 4.1.1 (Single R): The inter-request arrival time is a single scalar $R > 0$. That is, arrivals are both regularly spaced and item-independent. This idealisation follows the competitive-analysis convention of [4, 9].

Assumption 4.1.2 (Discrete times T): Times T are assumed discrete time-steps, i.e. $T \subseteq \mathbb{N}_0$. This is standard convention in caching competitive analysis [4, 9, 16].

4.2 Cache Networks

In this section, the single-cache model is expanded upon into a cache network. In general, this is a series of inter-connected caches and a backing store. Each network will have at least one 'ingress' or 'leaf' cache to which requests are sent, and there should be a path from every cache in the network to the backing-store. Definition 4.2.1 provides the *general* definition of a cache network.

Definition 4.2.1 (Cache Networks): Let $\mathcal{N}$ be a graph s.th. $\mathcal{N} = (V, E)$, where V is the set of vertices in the graph and $E \subseteq V \times V$ is the set of edges between two vertices in the graph. Then, $\mathcal{N}$ is a 'cache network' when:

1. Every vertex $v_x \in V$ represents a single cache $C_t^{(x)}$ (as in Definition 4.1.2);
2. There is a backing-store B connected to at least one cache vertex in the network;
3. There exists a valid path from any vertex to B; and
4. The network has at least one designated 'ingress' vertex (also referred to as a 'leaf') to which requests are initially sent.

Further, the total capacity of a cache network is $k_T = |V|k$, where k is the capacity of each cache (Definition 4.1.2). Finally, let $N \subset \mathcal{I}$ be the set of all items stored in the cache network $\mathcal{N}$ at any given time.

Ordinarily, when a request $r = (i, t)$ arrives at the leaf vertex v_0, if $h_0(i) = 0$ (note here $h_x(i)$ is the hit-indicator function (Definition 4.1.3) for cache $C_t^{(0)}$), then r is forwarded upstream to another cache-vertex v_x [13]. In this way, a network can be thought of as being made of multiple cache-backing-store pairs, connected in series. This justifies the introduction of the *inter-cache delay parameter*, Z_A, which is defined below:

Definition 4.2.2 (Inter-Cache Delay Parameter): Given a cache network $\mathcal{N}$, as in Definition 4.2.1, the inter-cache delay parameter is the delay parameter between two directly connected cache-vertices. That is, $\forall (v_x, v_y) \in E$ the latency incurred by forwarding a request from v_x to v_y is Z_A (Definition 4.1.4). The total delay between the request's ingress vertex and B is $Z_B = Z_A + \cdots + Z_A = |P|Z_A$, where $|P|$ is the number of caches in the path from ingress to B.

An important modelling choice is whether to classify hits at caches other than the ingress as *misses* or *hits*. If an item is located at a non-ingress cache, then it still incurs some delay dZ_A, indicating that it should perhaps be classified as a miss. However, the item was still located *somewhere* in the network, indicating a hit. Both interpretations have merit, and neither changes the technical result or the analysis, however it is important to state which is used to avoid confusion. Here, they are classified as hits. Specifically, 'non-zero-latency-hits' (NZLHs):

Definition 4.2.3 (Non-Zero-Latency-Hits): For some request $r_j = (i, t)$, if i is found at some cache-vertex v_x s.th. v_x is *not* an ingress, then this is still classified as a *hit* (i.e. $h_x(i) = 1$). This may sometimes be referred to as a 'non-zero-latency-hit', as it still incurs a latency of dZ_A, where d is the number of times a request is forwarded upstream until either it hits or reaches B.

A cache network processes requests in the following method [13]:

1. Request r_k for item $i \in \mathcal{I}$ arrives at time t to cache-vertex v_0.
2. If $h(i) = 1$ (hit) then v_0 returns the item, incurring latency $l_k = 0$.
3. If $h(i) = 0$ (miss), then v_0 forwards the request upstream, incurring a latency of $l_k = Z_A$. If v_0 can forward the request to many caches, then a *deterministic* algorithm must choose to which cache-vertex the request is forwarded.
4. Repeat from step 2. Each time, $l_k \leftarrow l_k + Z_A$ until B is reached or $h(i) = 1$, at which point $l_k = d \cdot Z_A$, where d is the length of the request's path in the network.

Cache-networks, as a whole, are governed by *content placement policies* (CPPs), also known as *decision policies* [8]. This is similar to how single-cache systems are governed by *eviction policies*. The CPP determines *which* cache a given item is admitted to [8]. Then, that cache's specific eviction policy is triggered to determine which item the cache will evict in order to admit i. Common CPPs include DEMOTE and LCE [8].

In the process provided above, if the request is eventually forwarded to B, B returns item i to the cache network. The eviction policy and CPP are applied to determine the redistribution of items across the caching network.

4.2.1 The LCE Placement Policy

Formally, the LCE policy copies the missed item (now returned by a fetch from B) to *every* cache that was queried.

4.2.2 Cache Line Networks

Definition 4.2.1 defines a *general* cache network of any topology. However, for the purposes and scope of this project, this definition has been reduced to *line-networks* in Definition 4.2.4, a subset of all general networks.

Algorithm 3: LEAVE-A-COPY-EVERYWHERE

Data: The request $r_j = (i, t)$ that caused i to be fetched from B and a set $S \subseteq V$ of queried caches in $\mathcal{N}$
Result: N/A
for $s \in S$ **do**
  $C^{(s)}_{t+Z_B+1} \leftarrow \text{update}(C^{(s)}_{t+Z_B}, r_j, \text{victim})$
end
return $C^{(s)}_{t+Z_B+1}$

Definition 4.2.4 (Cache Line Networks): A cache network $\mathcal{N}$, as in Definition 4.2.1, is a *line network* if:

1. Every vertex is connected to, at-most, two other vertices;
2. There is exactly one vertex that is connected to B, and this vertex is connected to exactly one other non-B vertex; and
3. There is exactly one ingress vertex (the 'leaf'). The leaf is connected to exactly one other vertex.

A line network $\mathcal{N}$ has an overall delay parameter of $Z_B = |V| Z_A$.

Figure 4.2 shows a general line network.

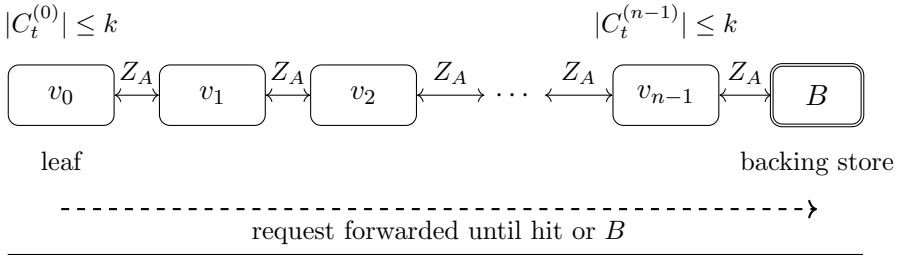


Figure 4.2: A line network of $|V|$ caches

This makes line-networks a very clearly defined subset of cache networks, and provides some key differences between them that restrict system behaviour and make competitive analyses more tractable to solve in the current amount of time allocated. Notably, the fact that every cache is connected to no more than two caches, with the leaf only being connected to one; and the fact that there is only *one* ingress cache (the leaf). This means that there is only exactly one path that any request can take, which

vastly constricts the space in which any cache placement policy has to work: instead of wondering both at which *branch and* at which cache in the branch an item should be placed, it only has to consider the latter. This difference will further explored in Chapter 7 (regarding tree topologies).

4.2.3 Assumptions for Cache (Line) Networks

This section outlines assumptions made when analysing caching line networks in this project.

Assumption 4.2.1 (Homogenous Z_A): $\forall(v_x, v_y), (v'_x, v'_y) \in E$, each with an inter-cache delay of Z_A and Z'_A, respectively, for a given network $\mathcal{N}$ (Definition 4.2.1), $Z_A = Z'_A$.

Assumption 4.2.2 (Homogenous k): $\forall v_x \in V$, $|C^{(x)}| = k$

4.3 Competitive Analysis for Caching

With the caching, the line network, and latency now defined, the central metric of this project may be formalised: the *competitive ratio* (CR). This measures an online policy [1] against the optimal offline policy [2].

Definition 4.3.1 (Competitive Ratio (Closed Form)): Given an online caching policy $\mathcal{P}$ and an offline optimal policy $\mathcal{O}$, the CR of $\mathcal{P}$ in a delayed-hits-aware context ($Z \geq 1$) is

$$\mathrm{CR}(\mathcal{P}, \rho) = \frac{L(\mathcal{P}, \rho)}{L(\mathcal{O}, \rho)},$$

where $L(\mathcal{P}, \rho) = \sum_{r_j \in \rho} l_j$ (Def 4.1) is the total latency incurred by $\mathcal{P}$ on $\rho = (r_j)_{j=1}^{N}$ [16].

From Definition 4.3.1, it can be seen that it is ideal to obtain $\mathrm{CR} = 1$ as this implies that $\mathcal{P}$ is as 'good' as $\mathcal{O}$ for the sequence ρ, with lower CRs being more optimal. However, this only describes the performance of $\mathcal{P}$ on a specific sequence ρ, not the performance of $\mathcal{P}$ *in general*. In order to describe it in general, the supremum over the sequence-space is used:

[1] An online policy is one that only knows past and current requests.

[2] An offline policy is one that can see the full request sequence, including future requests. Non-trivial offline policies are optimal ones.

Definition 4.3.2 (Competitive Ratio): Given an online caching policy $\mathcal{P}$ and an offline optimal policy $\mathcal{O}$, the CR of $\mathcal{P}$ in a delayed-hits-aware context ($Z \geq 1$) is

$$\mathrm{CR}(\mathcal{P}) = \sup_{\rho} \frac{L(\mathcal{P}, \rho)}{L(\mathcal{O}, \rho)},$$

where $L(\mathcal{P}, \rho) = \sum_{r_j \in \rho} l_j$ (Def 4.1) is the total latency incurred by $\mathcal{P}$ on $\rho = (r_j)_{j=1}^{N}$ [16].

Before continuing further, it is first important to understand fully what the CR shows: It is a description of the *worst-case* performance for $\mathcal{P}$. The lower-bound can then be defined as follows:

Theorem 4.3.1 (Competitive Ratio Lower-Bound): Let $\mathcal{P}$ be a deterministic online caching policy. Then $\exists \rho_A \in \mathcal{R}$ such that

$$CR(\mathcal{P}, \rho_A) \geq c \cdot x \implies CR(\mathcal{P}) \geq c \cdot x,$$

for some constant $c > 0$ and a variable parameter x. Equivalently,

$$\mathrm{CR}(\mathcal{P}) = \Omega(x).$$

ρ_A is known as the 'adversarial sequence' [9, 15].

And the upper-bound is defined as:

Theorem 4.3.2 (Competitive Ratio Upper-Bound): Let $\mathcal{P}$ be a deterministic online caching policy. Then:

$$\forall \rho \in \mathcal{R},\ CR(\mathcal{P}, \rho) \leq c \cdot x',$$

for some constant $c > 0$ and a variable parameter x'. Equivalently,

$$\mathrm{CR}(\mathcal{P}) = O(x').$$

Policy $\mathcal{P}$ is then referred to as '$O(x')$-competitive' [15, 16].

Finally, the analytic objective of this project is defined. When both bounds match, i.e. when $x = x'$, it guarantees the competitive ratio. This is known as a *tightness guarantee*, and is the objective of the analysis.

In 2020, Manohar and Williams [9], proved the theoretical lower-bound for the CR of any deterministic policy acting in a DH-aware context:

Theorem 4.3.3 (Manohar & Williams's Lower-Bound): In a single-cache system, given a deterministic online eviction policy $\mathcal{P}$ and an offline eviction policy $\mathcal{O}$, the competitive ratio of $\mathcal{P}$ has a lower-bound of

$$\mathrm{CR}(\mathcal{P}) = \Omega(kZ),$$

where k is the size of the cache, and Z is the delay parameter.

4.4 Finding the Lower-Bound

From Theorem 4.3.1, the method for finding the CR lower-bound can intuitively be extracted, and is further reinforced by Manohar & Williams' method [9].

1. Construct an adversarial request sequence ρ_A that maximises $\frac{L(\mathcal{P},\rho_A)}{L(\mathcal{O},\rho_A)}$
2. Compute $\frac{L(\mathcal{P},\rho_A)}{L(\mathcal{O},\rho_A)}$ in closed form
3. Determine the asymptotic lower bound on $CR(\mathcal{P})$.

This method works because the competitive ratio is taken over the supremum of sequences. Thus, any valid lower-bound x for the closed-form CR for some sequence ρ, $\mathrm{CR}(\mathcal{P}, \rho)$ is *also* a valid lower-bound for the open-form CR $\mathrm{CR}(\mathcal{P})$. This is true because $\mathrm{CR}(\mathcal{P}) \geq \mathrm{CR}(\mathcal{P}, \rho)\ \forall\ \rho$. More formally:

If $\Omega(x)$ is the lower-bound for $\mathrm{CR}(\mathcal{P}, \rho)$ then $\mathrm{CR}(\mathcal{P}, \rho) \geq c \cdot x$ and

$$\mathrm{CR}(\mathcal{P}) \geq \mathrm{CR}(\mathcal{P}, \rho) \geq c \cdot x \implies CR(\mathcal{P}) \geq c \cdot x,$$

so $\Omega(x)$ is a valid lower-bound for $\mathrm{CR}(\mathcal{P})$ as well.

4.5 Finding the Upper-Bound

The upper-bound presents a more time-consuming challenge than the lower-bound does, as it is a universal claim across the sequence-space. Thus, while the lower-bound can be approached by iterating the 'development' of a single sequence, the upper-bound must be proven by making claims that hold for all sequences. Gurushankar et. al. [16] claim that partitioning a general sequence into phases is standard in competitive caching analysis, and then they further partition those phases into 'super-phases'. So, beginning with phase partitioning may be a useful first, experimental step to take.

Finding the upper-bound consists of *upper*-bounding $L(\mathcal{P})$ (the numerator) and *lower*-bounding $L(\mathcal{O})$ and then computing the CR with these bounded values [15, 16]. It should, however, be noted that *this* lower-bounding is distinct from the general

CR lower-bound. From Definition 4.3.1, the CR lower-bound is defined by *one* adversarial sequence input, whereas the lower-bound denominator found for $L(\mathcal{O})$, by Theorem 4.3.2, must hold for *all* possible sequences.

Chapter 5

Competitive Analysis of LCE on Line Networks

This section presents three main, novel contributions. The first is proof that the current lower-bound of $\Omega(kZ)$ does not hold for line networks with $|V|$ caches. The second is an LCE-based lower-bound for line networks. A theorem that shows that LCE us uncompetitive is outlined and proved. All three are preceeded by first adequately outlining the constraints of $\mathcal{O}$. This is outlined in Section 5.1: specifically, it outlines what $\mathcal{O}$ *can* and *cannot* do. Then, Chapter 5.2 contains the initial contribution done for this project. It formally explores how the sequence constructed by Manohar & Williams [9] behaves on a general line network. Next, Chapter 5.3 provides the first central contribution: A lower-bound on the LCE policy on a line network of $|V|$ caches. The same section also outlines the adversarial sequence that was constructed, and builds the intuition behind it. The second, and final, central contribution is found in Chapter 5.4. By first providing the formal theorem for uncompetitiveness, it is shown that the LCE policy is uncompetitive (unbounded) on a line network.

5.1 Choosing $\mathcal{O}$ For CPPs

In conventional literature, the $\mathcal{O}$ policy that is used is the optimal, offline, *eviction* policy. These are Belády's MIN algorithm in classical caching [11], and the BELATEDLY algorithm for DH-aware caching [4]. Such policies typically have the ability to evict whichever item they 'want' after each cache-miss. However, here, $\mathcal{O}$ represents the optimal, offline *CPP* [6]. It *cannot* determine which items are to be evicted from individual caches, as this is determined by the per-cache eviction policy. So, $\mathcal{O}$ should only be able to freely place any item that B returns, and any item evicted by any cache, to any cache at its discretion.

Further, optimal eviction policies implicitly gain admission policy properties. They may admit an undesirable item and then evict it immediately, reverting the cache to

its previous state. An optimal CPP, however, retains no such option because eviction is not under its control. Accordingly, $\mathcal{O}$ is granted the ability to decline placement outright.

5.2 Initial Lower-Bound

The first step for finding a lower-bound was to apply an existing adversarial sequence to a line-network. This was done as it would provide a baseline result against which to compare future iterations of the adversarial sequence. Furthermore, applying this sequence to a line network was a natural first step: it tested whether the existing adversary is topology-sensitive, thereby justifying it as a baseline result.

The existing sequence chosen was the one constructed by Manohar & Williams ρ_{MW}, which yielded a lower-bound of $\Omega(kZ)$ on DH-aware, single-cache systems. This sequence is outlined below, along with the terminology required to understand it.

By convention, single caches are typically initialised with the first k items in the universe $\mathcal{I}$ [9]. The same convention has been continued here, in networks. The cache line network $\mathcal{N}$ is initialised with the first k_T items in $\mathcal{I}$, like so:

$$\begin{aligned} C_0^{(0)} &= \{i_1, i_2, \ldots, i_k\}, \\ C_0^{(1)} &= \{i_{k+1}, i_{k+2}, \ldots, i_{2k}\}, \\ &\ldots, \\ C_0^{(|V|-1)} &= \{i_{(|V|-1)k+1}, i_{(|V|-1)k+2}, \ldots, i_{k_T}\} \end{aligned}$$

Note that for readability purposes, the corresponding τ-value for each item in each cache has been omitted here.

Before providing ρ_{MW}, two special types of requests must first be formally defined. These are *bursty* and *pure* requests:

Definition 5.2.1 (Bursty Request): Given a sequence of requests $\rho = (r_j)_{j=1}^{j=N}$ (Definition 4.1.1) and a delay parameter Z_B, a bursty request is a subsequence of ρ, and is represented by a Q-tuple,

$$\beta = (b_q)_{q=1}^{q=Q},$$

s.th. requests:

- $b_1, \ldots, b_{Z_B}$ are non-requests (i.e. for $\perp$);
- $b_{Z_B+1}, \ldots, b_{2Z_B}$ are for some item $i \in \mathcal{I}$; and
- $b_{2Z_B}, \ldots, b_Q$ are again for non-requests (so $Q = 3Z_B$).

Further any request that is not for $\perp$ is referred to as an 'actual' request. Note that all actual requests in β are for the *same* item.

Definition 5.2.2 (Pure Request): Given a sequence of requests $\rho = (r_j)_{j=1}^{j=N}$ (Definition 4.1.1) and a delay parameter Z_B, a pure request is a subsequence of ρ, and is represented by a Q-tuple,

$$\pi = (p_q)_{q=1}^{q=Q},$$

s.th. requests:

- $b_1, \ldots, b_{Z_B}$ are non-requests (i.e. for $\perp$);
- b_{Z_B+1} is for some item $i \in \mathcal{I}$; and
- $b_{Z_B+2}, \ldots, b_Q$ are again for non-requests.

These bursty and pure requests are useful tools for adversarial sequence construction as they are 'self-contained' in that delayed-hits for the item requested only exist for however long the bursty or pure request lasts. Because the last (or only) request occurs just before Z_B time-steps of no requests, all outstanding fetch requests will be resolved by the time the overall bursty or pure request ends.

Using these bursty and pure requests, ρ_{MW} is redefined within the context of a *cache network*:

1. Make a pure request for $i \in \mathcal{I}$ s.th. $i \notin N$ (that is, i is not in any cache in the network).
2. Make a bursty request for $i \in \mathcal{I}$ s.th. $i \notin N$.
3. Repeat step 2 $k_T - 1$ more times.

The analysis for the lower-bound on a line network using ρ_{MW} relies on two supporting lemmas which are then used in the main proof (Theorem 5.2.1). Throughout, $\mathcal{P}$ is a deterministic, online CPP, and $\mathcal{O}$ is the optimum, offline CPP.

Lemma 5.2.1 ($\mathcal{P}$'s Latency): $\mathcal{P}$ incurs a latency of $L(\mathcal{P}, \rho_{\text{MW}}) \geq Z_B + k_T \cdot \frac{Z_B(Z_B+1)}{2}$.

Proof. The first request $r_1 = (i, 1)$ is a pure request (Definition 5.2.2). Because the requested item has been chosen s.th. $i \notin N \implies h_x(i) = 0 \; \forall \; x \in V$. The request misses at all caches and so is classified as a cache-miss, and is fetched from B. Thus, $l_1 = Z_B$.

At $1 + Z_B$, when r_1's item is fetched, $\mathcal{P}$ may rearrange the network. Depending on the policy, item i may be rejected, or $\mathcal{P}$ may order one or all of the caches to evict an item. Regardless, after $t = 1 + Z_B$, there will be at least one item that is *not* in N. This is true for all t as $|\mathcal{I}| \geq |V|k$ (non-trivial)

Following r_1, there are k_T bursty requests. Each request in a burst is for the item not in N at the beginning of that burst. Let β_g represent the gth bursty request in ρ_{MW} (Definition 5.2.1). As stated previously, β_g begins with Z_B requests for $\perp$ (no item), followed by Z_B requests for some item i, followed by Z_B requests for $\perp$. In general, if the first actual request of a bursty request misses, then the following ones are delayed-hits. In this specific case, the first actual request is *always* a cache-miss. This is guaranteed as the adversary has chosen to request an item that is not in the network. Thus, requests $b_{Z_B+1}, \ldots, b_{2Z_B}$ incur a collective latency of $Z_B + (Z_B - 1) + \cdots + 1 = \frac{Z_B(Z_B+1)}{2}$ [9]. Finally, since there are k_T bursty requests, the 'bursty latency' of the sequence is

$$k_T \cdot \frac{Z_B(Z_B+1)}{2},$$

giving:

$$L(\mathcal{P}, \rho_{\text{MW}}) \geq Z_B + k_T \cdot \frac{Z_B(Z_B+1)}{2}.$$

□

Remark (Reparametrisation): Lemma **??** is precisely the result of Manohar & Williams ($L(\mathcal{P}, \rho_a) \geq Z + kZ\frac{Z+1}{2}$) [9], under the reparametrisation:

- $k \mapsto k_T$; and
- $Z \mapsto Z_B$.

Lemma 5.2.2 ($\mathcal{O}$ Hits on all Bursties): $\mathcal{O}$ hits on all bursty requests in ρ_{MW}.

Proof. The marking argument of Manohar & Williams [9] depends only on the cache holding a fixed number of distinct items; it is insensitive to whether that capacity is realised as one cache or as multiple (i.e. a network). Replacing the single-cache capacity k with the network capacity $k_T = |V|k$ therefore leaves the argument unchanged: after the initial fetch, $\mathcal{O}$ admits every requested item but the one unmarked $j \in \{1, \ldots, k_T + 1\} \setminus M$ and never rearranges again, so no bursty request results in a full miss. Hence $\mathcal{O}$ hits on every bursty request. □

Lemma 5.2.3 ($\mathcal{O}$'s Latency): $\mathcal{O}$ incurs a latency of $L(\mathcal{O}, \rho_{\text{MW}}) = Z_B + kZ_B^2 \cdot \frac{(|V|-1)}{2}$

Proof. Again, the first pure request $r_1 = (i, 1)$ incurs a miss for the same reason as in Lemma 5.2.1. This is because, although $\mathcal{O}$ knows what r_1 is requesting prior to the request arriving, it is unable to rearrange the network in response as it can only rearrange when B returns a fetch request. Thus, $l_1 = Z_B$.

Once again, at $t = 1 + Z_B$, $\mathcal{O}$ has the opportunity to rearrange the network as item i has been returned from B. From Lemma 5.2.2, the $\mathcal{O}$ line network hits on all bursty requests (as is also the case in [9]). The key difference here, though, is that such hits are not necessarily zero-latency.

Every hit incurs a latency of $d \cdot Z_A$ (Definition 4.2.3), where d is the number of times the request was forwarded upstream. From Lemma 5.2.2, after $t = 1 + Z_B$ (i.e. the return for the item requested in r_1), every item that will be requested in ρ_{MW} is already stored in N. Further, after r_1 there are k_T requests for distinct items,

$$\forall i \in N, \exists r_j \in \rho_{\text{MW}} \text{ s.th. } r_j = (i, t)$$

Let d be the number of forwards until a hit for a specific item $i \in N$. Then, for each bursty request β_g,

$$l_g = Z_B \cdot Z_A d.$$

Finding the total latency of all bursty requests is equivalent to summing $k(Z_B Z_A d)$ over the number of caches:

$$L_B = kZ_B Z_A \sum_{d=1}^{|V|-1} d = kZ_B Z_A \cdot \frac{(|V|-1)|V|}{2}$$

Substituting $Z_B = |V|Z_A$ (Definition 4.2.4):

$$L_B = kZ_B^2 \cdot \frac{(|V|-1)}{2}$$

Finally:

$$L(\mathcal{O}, \rho_{\text{MW}}) = Z_B + kZ_B^2 \cdot \frac{(|V|-1)}{2}$$

□

These lemmas are then utilised in the final theorem, showing the initial lower-bound of CPPs on line-networks using ρ_{MW}.

Theorem 5.2.1 (Initial Lower-Bound): In a cache line network $\mathcal{N}$ (Definition 4.2.4), given a deterministic online placement policy $\mathcal{P}$ and an offline eviction policy $\mathcal{O}$, the competitive ratio of $\mathcal{P}$ has a lower-bound of

$$\text{CR}(\mathcal{P}) = \Omega(1).$$

Proof. From Lemma 5.2.1 and Lemma 5.2.3, respectively,

$$L(\mathcal{P}, \rho_{\text{MW}}) \geq Z_B + k_T \cdot \frac{Z_B(Z_B+1)}{2},$$
$$L(\mathcal{O}, \rho_{\text{MW}}) = Z_B + kZ_B^2 \cdot \frac{(|V|-1)}{2}$$

Then, from Definition 4.3.1:

$$\text{CR}(\mathcal{P}, \rho_{\text{MW}}) \geq \frac{Z_B + k_T \cdot \frac{Z_B(Z_B+1)}{2}}{Z_B + kZ_B^2 \cdot \frac{(|V|-1)}{2}}$$
$$\implies \text{CR}(\mathcal{P}, \rho_{\text{MW}}) \geq \frac{2 + k|V|^2 Z_A + k|V|}{2 + k|V|^2 Z_A - k|V|Z_A}$$

Then, as $|V|, k, Z_A \to \infty$:

$$\text{CR}(\mathcal{P}, \rho_{\text{MW}}) \geq \frac{k|V|^2 Z_A}{k|V|^2 Z_A - k|V|Z_A}$$
$$\text{CR}(\mathcal{P}, \rho_{\text{MW}}) \geq \frac{k|V|^2 Z_A}{k|V|Z_A(|V|-1)}$$
$$\text{CR}(\mathcal{P}, \rho_{\text{MW}}) \geq \frac{|V|}{|V|-1}$$
$$\text{CR}(\mathcal{P}, \rho_{\text{MW}}) \geq 1$$

Finally, using Theorem 4.3.1:

$$\begin{aligned} &\mathrm{CR}(\mathcal{P}) \geq \mathrm{CR}(\mathcal{P}, \rho_{\mathrm{MW}}) \geq 1 \\ \implies &\mathrm{CR}(\mathcal{P}) = \Omega(1) \end{aligned}$$

□

Evaluation of Result. The sequence constructed by Manohar & Williams does not generalise to a cache network while measuring a CPP. The resulting bound is $\Omega(1)$, which is trivial by definition. This contrasts sharply with the single-cache case, where the same sequence yields $\Omega(kZ)$. This can be attributed to the fact that $\mathcal{O}$ incurs latency costs as heavily as $\mathcal{P}$ during cache-hits. This is corroborated by $L(\mathcal{P}, \rho_{\mathrm{MW}})$, which is identical to the single-cache result. The adversary treats the network as one large cache, rather than as multiple inter-connected ones. It should be noted, however, that Manohar & Williams' original paper is for a lower-bound for the single-cache eviction policy, not the CPP (as the CPP is trivial in the case of $|V| = 1$).

5.3 Lower-Bounding LCE

Following the initial trivial result of the preceding section, the next step in finding the lower-bound was to use the lessons learnt from the ρ_{MW} lower-bound $\Omega(1)$ and apply them to construct a new sequence. The specific issue that resulted in a trivial lower-bound was that $\mathcal{O}$ incurred latency as 'heavily' as $\mathcal{P}$ did. Thus, the new sequence ρ_A should be constructed such that $\mathcal{O}$ incurs *far less* latency than $\mathcal{P}$. As will be seen below, the best-found way to achieve this is to ensure all hits are shallow hits for $\mathcal{O}$, so that they incur minimum or even zero latency.

5.3.1 Adversarial Sequence for LCE

Before outlining the adversarial sequence that has been constructed in order to yield the required lower-bound, a set W must be defined. This will act as a useful tool to bound the sequence to ensure its finiteness, as well as also ensuring $\mathcal{O}$ achieves shallow hits.

Definition 5.3.1 (The Set W): Given a line network with $|V|$ caches, each of capacity k, let W be a set s.th.

$$W \subset \mathcal{I} \wedge |W| = k + 1$$

Remark (Significance of W): By literature convention [4,9,16], a single cache system is initialised with the first k-distinct items of a size-ordered sequence of $\mathcal{I}$. Thus, exactly k items in W reside in $C^{(0)}$ of the line network, and one item at $C^{(1)}$.

Then, an example 'toy' network is outlined below which will be used to initially build the intuition for the sequence constructed in Definition 5.3.2. This is known hereinafter as the $(2,2)$-Network, or $\mathcal{N}_{(2,2)}$

Example 5.3.1 (The (2,2) Network): A line network with $|V| = 2$, $k = 2$, $Z_A = 2$ ($Z_B = 4$) and $\mathcal{I} = \{a, b, c, d, e, f\}$, $W = \{a, b, c\}$, TTL $= 6$, is used as a trace throughout this analysis. At $t = 0$, $v_0 = \{(a, 6), (b, 6)\}$, $v_1 = \{(c, 6), (d, 6)\}$.

The adversarial sequence is defined below.

Definition 5.3.2 (Adversarial Sequence ρ_A): Given a line network $\mathcal{N}$ with $|V|$ caches, each of capacity k and using the TTL eviction policy, and a universe of items $\mathcal{I}$, let ρ_A,

1. Make a pure requests for any item not in N at $t = 1$.
2. Wait / make $\perp$-requests for Z_B time steps.
3. Make a bursty request β_g for an item $i \in \mathcal{I}$ s.th. $i \in W \wedge i \notin N$ at $t = 5$.
4. Repeat step two $|W| - 1$ more times.

be an adversarial sequence used on $\mathcal{N}$.

Remark (ρ_A on the $(2,2)$-Network): On the $(2,2)$-Network shown in Example 5.3.1, ρ_A can reduce to:

$$\rho_A = (e/f, \beta_1, \beta_2, \beta_3),$$

where β_1 requests item a, β_2 item c, and β_3 item b. Note that there are other configurations for the reduced ρ_A.

Because $|V|k < |\mathcal{I}|$ for any non-trivial cache network, there exists at least *one* $i \in \mathcal{I}$ s.th. $i \notin N$ at $t = 0$. Request this item at $t = 1$. Since $\mathcal{I} \setminus N = \{e, f\}$, requesting e or f results in a full miss and a fetch request (resolving at $t = 5$). At this point, it could seem beneficial to take advantage of DHs and continuously request e for 'free' latency. However, this would inadvertently disadvantage $\mathcal{O}$. Neither $\mathcal{O}$ or $\mathcal{P}$ can rearrange the

network *before* the first fetch return. Thus, during this time, any latency that $\mathcal{P}$ incurs will also be incurred by $\mathcal{O}$, which is counter-productive. So, only one, single request is sent (e or f here), rather than multiple for the same item.

At $t = 5$ the fetch for e/f returns. The state of the network at this time is $C_4^{(0)} = \{(a, 1), (b, 1)\}, C_4^{(1)} = \{(c, 1), (d, 1)\}$. LCE mandates that e/f (take e from now on) is admitted at every cache, so the eviction policy evicts the item with the lowest TTL in each cache, thus providing $C_5^{(0)} = \{(e, 6), (b, 1)\}, C_5^{(1)} = \{(e, 6), (d, 5)\}$. Now, each element of W is continuously requested (via bursty requests) in an order such that all bursty requests incur misses. This is possible as, after $t = 5$, there is always at least one item in W that is not in the network. Thus, for this trace, the final adversarial sequence on $\mathcal{N}_{(2,2)}$ is:

$$\rho_A = (e, a^{Z_B}, c^{Z_B}, b^{Z_B})$$

5.3.2 LCE Lower-Bound

As with ρ_{MW}, the final theorem (Theorem 5.3.1 that provides the lower-bound of ρ_A on a line network also relies on two preliminary lemmas.

> ***Lemma 5.3.1 (LCE's Latency):*** LCE incurs a latency of $L(\text{LCE}, \rho_A) \geq \frac{1}{2}kZ_B^2 + \frac{3}{2}Z_B + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2$.

Proof. As previously, r_1 incurs a miss as $k_T \leq |\mathcal{I}|$, so $l_1 = Z_B$.

Then, at $t = 5$, the item for r_1 has been fetched and the network admits it under LCE. Now all caches in $\mathcal{N}$ have this item, and have evicted one item each under TTL. Due to the way W has been constructed (Definition 5.3.1), there is at least one item $i \in W \wedge i \notin N$ which has also not yet been requested. The subsequent bursty request β_1 requests that item. As shown in Lemma 5.2.1, bursty requests that incur all misses yields a latency of $\frac{Z_B(Z_B+1)}{2}$, and since there are $k + 1$ bursty requests:

$$\begin{aligned} l_B &= (k+1) \cdot \frac{Z_B(Z_B+1)}{2} \\ &= \frac{Z_B(k+1)(Z_B+1)}{2} \\ &= \frac{kZ_B^2 + kZ_B + Z_B^2 + Z_B}{2} \end{aligned}$$

Giving:

$$\begin{aligned} L(\mathcal{P}, \rho_A) &\geq Z_B + l_B \\ L(\mathcal{P}, \rho_A) &\geq Z_B + \frac{1}{2}kZ_B^2 + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2 + \frac{1}{2}Z_B \\ &\geq \frac{1}{2}kZ_B^2 + \frac{3}{2}Z_B + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2 \end{aligned}$$

□

Lemma 5.3.2 ($\mathcal{O}$ Hits on all Bursties): $\mathcal{O}$ hits on all bursty requests in ρ_A.

Proof. All bursty requests $\beta_1, \beta_2, \beta_3$ (Definition 5.3.2) are for items that are already in the cache at $t = 5$ (when r_1's item returns) by Definitions 5.3.2 and 5.3.1. Thus, all bursty requests will incur cache-hits. □

Remark (NZLHs on $\mathcal{O}$): However, although some will be NZLHs (Definition 4.2.3), $\mathcal{O}$ cannot do better than this by rearranging as there will always be exactly one item that is burstily-requested that is not in the leaf cache.

Lemma 5.3.3 ($\mathcal{O}$'s Latency): $\mathcal{O}$ incurs a latency of $L(\mathcal{O}, \rho_A) = Z_B + Z_B \cdot Z_A$

Proof. Like with LCE in Lemma 5.3.1, $\mathcal{O}$ has a full cache-miss with r_1 and thus incurs $l_1 = Z_B$.

From Lemma 5.3.2, $\mathcal{O}$ experiences all cache-hits on the $\beta_1, \beta_2, \beta_3$ bursty request sequences (Definition 5.3.2). It only remains to show the latency incurred by these hits, as some of them are, again, NZLHs.

Due to the way W is defined (Definition 5.3.1) and the way ρ_A is constructed (Definition 5.3.2), $\mathcal{O}$ has exactly k zero-latency hits and one NZLH. This NZLH is guaranteed to be at vertex v_1's cache. Thus, the full bursty portion of ρ_A incurs $Z_B Z_A$ on ρ_A (as there are Z_B requests for the item at v_1's cache).

Putting this together yields:

$$\begin{aligned} L(\mathcal{O}, \rho_A) &= Z_B + l_B \\ &= Z_B + Z_B \cdot Z_A \end{aligned}$$

□

Theorem 5.3.1 (LCE Lower-Bound): In a cache line network $\mathcal{N}$ (Definition 4.2.4), given the LCE placement policy LCE and an offline eviction policy $\mathcal{O}$, the competitive ratio of LCE has a lower-bound of

$$\mathrm{CR}(\mathrm{LCE}) = \Omega(k|V|).$$

Proof. From Lemma 5.3.1 and Lemma 5.3.3, respectively,

$$\begin{aligned} L(\mathrm{LCE}, \rho_A) &\geq \frac{1}{2}kZ_B^2 + \frac{3}{2}Z_B + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2, \\ L(\mathcal{O}, \rho_{\mathrm{MW}}) &= Z_B + Z_B \cdot Z_A \end{aligned}$$

Then, from Definition 4.3.1:

$$\begin{aligned} \mathrm{CR}(\mathrm{LCE}, \rho_A) &\geq \frac{\frac{1}{2}kZ_B^2 + \frac{3}{2}Z_B + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2}{Z_B + Z_B \cdot Z_A} \\ \implies \mathrm{CR}(\mathrm{LCE}, \rho_A) &\geq \frac{2 + k|V|^2 Z_A + k|V|}{2 + k|V|^2 Z_A - k|V|Z_A} \end{aligned}$$

Then, as $|V|, k, Z_A \to \infty$:

$$\begin{aligned} \mathrm{CR}(\mathrm{LCE}, \rho_A) &\geq \frac{\frac{1}{2}kZ_B^2}{Z_B \cdot Z_A} \\ \mathrm{CR}(\mathrm{LCE}, \rho_A) &\geq \frac{kZ_B^2}{2} \cdot \frac{1}{Z_B Z_A} = \frac{kZ_B}{2Z_A} \end{aligned}$$

Finally, substituting in $Z_B = |V|Z_A$:

$$\begin{aligned} CR(\mathrm{LCE}, \rho_A) &\geq \frac{k|V|Z_A}{2Z_A} \\ &\geq \frac{k|V|}{2} \end{aligned}$$

Finally, using Theorem 4.3.1:

$$\begin{aligned} \mathrm{CR}(\mathrm{LCE}) &\geq \mathrm{CR}(\mathrm{LCE}, \rho_A) \geq \frac{k|V|}{2} \\ \implies \mathrm{CR}(\mathrm{LCE}) &= \Omega(k|V|) \end{aligned}$$

□

Evaluation of Result. The result of $\Omega(k|V|)$ is far less trivial than the result in Section 5.2, as it scales with both the per-cache capacity and the network length. The

objective here was to find a bound that grew with the network-dependent parameters, and this has been achieved, as the bound is defined in terms of both k and $|V|$.

5.4 LCE has no Upper-Bound

Following the proof that LCE has a lower-bound of $\Omega(k|V|)$, the next step in the theoretical side of the project was to find a matching upper-bound, or to prove that LCE was uncompetitive. The latter was, in the end, proven. This section formally defines the property of uncompetitiveness, and then discusses (and subsequently proves) such a property is true for the LCE CPP.

It should first be noted, however, that common upper-bound analytic techniques described in Chapter 3 were explored further and deeply before arriving at this possibility. Specifically, the possibility of determining whether Gurushankar et al.'s [16] phase partitioning analysis for DH-aware single-cache systems holds for line-networks, with or without modifications. However, due to restrictions on how the marking property constrains the search space for an item to evict, and the fact that hits are no longer 'free' on line-networks, this line was not compatible with the networks being studied. Another branch that was investigated briefly was the possibility of using Primal-Dual linear programming [20] to help in determining a sufficient upper-bound. In actuality, the correct direction was to show that LCE is uncompetitive on line-networks, the findings of which are outlined below.

As stated in Chapter 3, some policies may not be competitive in that their upper-bounds do not grow with the prescribed variables (namely k, $|V|$, $|\mathcal{I}|$ and Z_A here) [22, 23]. Trivially, of course, the upper-bound will always dependent on the size of the input sequence.

Theorem 4.3.2 identifies the CR upper-bound. For completeness, it is informally restated here. It specifies that, **for all** input sequences,

$$\mathrm{CR}(\mathcal{P}, \rho) \leq c \cdot x'$$

for some constant $c > 0$ and a variable parameter x'. This gives rise to the following corollary:

> ***Corollary 5.4.1 (Uncompetitiveness):*** A deterministic policy $\mathcal{P}$ can be proven to be uncompetitive iff $\exists\{\rho_n\}$ s.th. $\mathrm{CR}(\mathcal{P}, \rho_n) \to \infty$, with x' fixed, where $\{\rho_n\}$ is a family of sequences [23].

Proof. Immediate from Theorem 4.3.2. If there exists a sequence-family for which the bound $\mathrm{CR}(\mathcal{P}, \rho_n) \leq c \cdot x'$ does not hold, then that policy violates the theorem, thus showing that it is not competitive. That is, a sequence which results in $\mathrm{CR}(\mathcal{P}, \rho_n) \to \infty$ (for some fixed x'). □

So, it remains to then identify a family of sequences that proves Corollary 5.4.1 is true for LCE.

5.4.1 A Family of Sequences That Violate Theorem 4.3.2

Let ϱ be a sequence in the family $\{\rho_n\}$. In order to prove Corollary 5.4.1, $L(\text{LCE})$ should grow much faster than $L(\mathcal{O})$ on a given sequence. On a line-network $\mathcal{N}$, this can be done by repeatedly requesting a specific item g that is located deeper than the leaf vertex cache. Since this will technically count as a cache-hit (an NZLH), LCE will have no opportunity to rearrange the network. It will repeatedly incur a latency of dZ_A for however many requests for g are sent, as long as there are no intermittent cache-misses. Thus, ϱ must accomplish three things:

1. It must, given an item $g \in \mathcal{I}$, adversarially request items in such a way that g is *only* placed at $C_t^{(x)}$ s.th. $x \neq 0$;
2. It must set g such that $g \notin N$ at $t = 0$, giving $\mathcal{O}$ an opportunity to rearrange; and
3. It must then request g repeatedly, for an arbitrary amount of time.

For ease of explanation, ϱ is now split into two subsequences,

$$\varrho = \varphi + \mu,$$

whereby φ will be designed to complete points (1) and (2), and μ to complete (3). Below, algorithms to construct each subsequence are presented, along with their intuitions.

First, φ will be constructed. Begin by identifying g as any item s.th. $g \notin N$ at $t = 0$. Then, make one pure request (Definition 5.2.2) for g. This will cause LCE to place g at every cache in the network, including at v_0. Following this, in an iterative manner, identify the shallowest cache that has g, and request every item other than g until g is evicted from that cache. Repeat this, at-most, until g is only present in the $|V| - 1$th cache.

After φ, cache-hits at g should incur some sort of latency. Then, μ must be defined: Request g an arbitrary number of times. The lengths of ϱ, φ, and μ are denoted as N_ϱ, N_φ, and N_μ, respectively.

Finally, it remains to prove that LCE is not competitive due to the existence of $\{\rho_n\}$.

Theorem 5.4.1 (LCE is Not Competitive): Given a line-network $\mathcal{N}$ (Definition 4.2.4), the LCE placement policy is not competitive. That is, given the family of sequences $\{\rho_n\}$,

$$\mathrm{CR}(\mathrm{LCE}, \rho_n) \to \infty$$

for some fixed k, $|V|$, and Z_A, and for $N_\mu \to \infty$, as per Corollary 5.4.1.

Proof. LCE Analysis. By design, the adversary constructed μ such that it requests an item that is in N but not in $C_t^{(0)}$ (the only zero-latency-incurring cache). Thus, each $g \in \mu$ incurs a latency of dZ_A (Definition 4.2.3). In total, then, LCE incurs a latency $\geq dZ_A \cdot N_\mu$.

$\mathcal{O}$ Analysis. After the initial fetch for g, $\mathcal{O}$ may rearrange the cache. It places g in the 0th cache, ensuring that all requests in μ incur zero latency.

N_φ is determined by k and $|V|$ alone. Both $L(\mathrm{LCE}, \varphi)$ and $L(\mathcal{O}, \varphi)$ are, therefore, bounded by a constant independent of N_μ. Further, $L(\mathcal{O}, \mu)$ is invariant to N_μ as it incurs no latency from μ regardless of length. Therefore, $L(\mathcal{O}, \mu) = 0$.

Thus, because $L(\mathcal{O}, \mu) = 0$ and $L(\mathrm{LCE}, \mu) \geq dZ_A \cdot N_\mu$,

$$\mathrm{CR}(\mathrm{LCE}, \varrho) = \frac{L(\mathrm{LCE}, \varphi) + dZ_A \cdot N_\mu}{L(\mathcal{O}, \varphi)}$$

Only the growth of LCR's CR is dependent on N_μ, so as $N_\mu \to \infty$, $\mathrm{CR}(\mathrm{LCE}, \varrho) \to \infty$, meaning that the LCE policy is not competitive. □

In this chapter, various statements about how LCE's competitive ratio grows with key variables (k, $|V|$, and Z_A) have been made and proven. Importantly, it has been shown that, although it does have a lower-bound of $\Omega(k|V|)$, LCE is unbounded above – thereby proving uncompetitive. Further, this chapter proved that the ρ_{MW} sequence yields a trivial lower-bound of $\Omega(1)$, which motivated further work into finding a tighter bound. In the next chapter, such bounds will be verified by empirical simulations. The simulation framework's specific design philosophy is also outlined, and the results of the simulations are evaluated.

Chapter 6

Simulation Results

Here, the bounds found and proven in Chapter 5 will be empirically tested and verified. First, the framework that has been developed for simulating cache networks is outlined. This includes the design philosophy of the framework, and the specific stack of modules used. Next, all the adversarial sequences, ρ_{MW}, ρ_A, and ρ_n, were inputted into the simulation.

6.1 Framework Design

The framework developed for this project was made using the Python language. This language was chosen due to the fact that prototypes can be iterated on rather quickly, which suited the nature of the work: adversarial sequences were developed and built-upon from past iterations.

The framework and simulation was developed with an object-oriented and modular philosophy. Requests, Sequences, Clocks, Caches, and Networks were all modelled as separate classes that depended upon and called each other. Furthermore, the `dataclasses` and `typing` Python modules were heavily used. These allow a developer to more closely model mathematical structures and implement software in a more formal manner. They allow classes to be defined by their fields and field types alone, so formal definitions can be translated into code faithfully.

Another design choice made was to use one, universal clock rather than multiple for each object. In the initial iteration, each major module (e.g. each cache, the network, and the sequence) had their own clocks. This meant that, at each time-step, they had to be updated individually and at the exact same time. If they weren't, then synchronisation issues would occur and the simulation would fail. Ensuring all clocks were updated at the same time, identically created overcomplicated, hard-to-read code. Thus, the choice was made to have one master clock that 'handed' a reference of its personal time object to the clock of major components. This essentially acts as a pointer to the master clock itself. Thus, when the master clock updated, the internal

times in each module update automatically. This allowed the framework to maintain its modular design without creating overcomplicated timing code, as only one line for time-stepping was required.

6.1.1 Structures, Objects & Classes

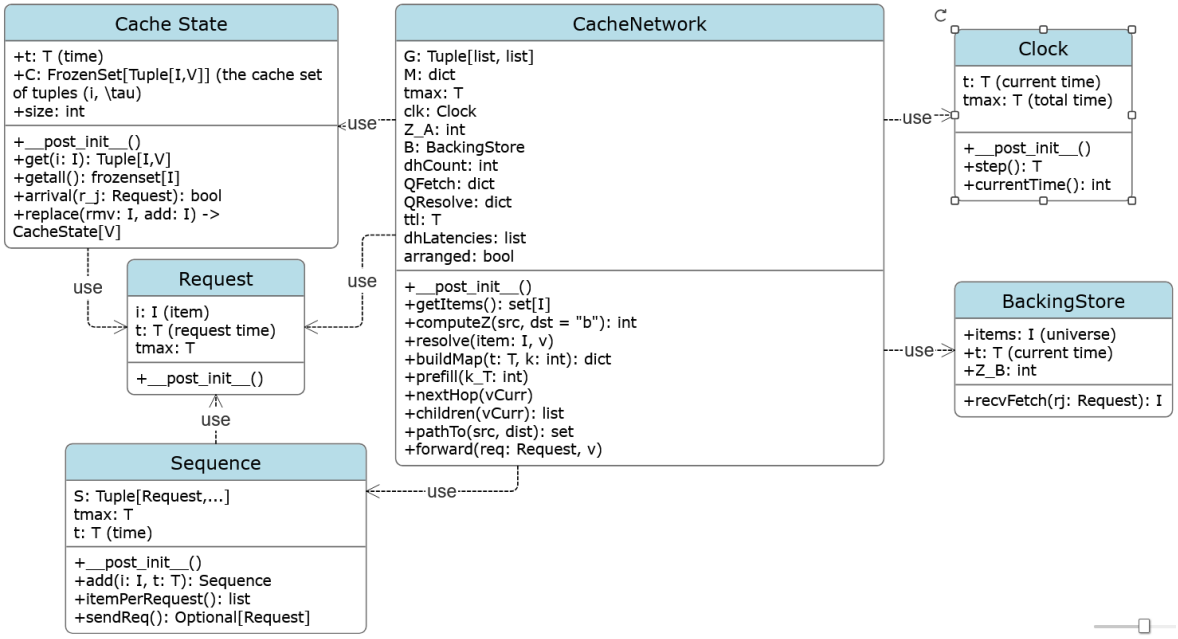


Figure 6.1: UML Class Diagram for Main Classes

Figure 6.1 provides a general layout for the core components used in the framework. It shows a UML class diagram that portrays the ways in which the `Sequence`, `Request`, `CacheState`, `Clock`, `BackingStore`, and `CacheNetwork` interact and their relations to each other. As stated previously, the framework was designed in a very modular fashion: even the way individual cache states are represented can be changed easily without having to change `CacheNetwork` much, if at all.

6.1.2 Generalised Algorithms

Furthermore, there was an effort to make the underlying framework generalised such that it can work on any arbitrary topology. That is, algorithms and methods such as forwarding requests upstream, determining a request's path from leaf to backing-store, and hopping to the next cache in the network, were written in a generalised manner: they do not assume anything about the topology of the provided network. Furthermore, the LCE CPP was written in the same manner. Relevant algorithms are provided below.

Algorithm 4: NEXT-HOP

Data: The network $\mathcal{N} = (V, E)$; the current vertex $v \in V$.
Result: The vertex to which a request resident at v is forwarded.
$P \leftarrow \{ u : (v, u) \in E \wedge u \neq v' \}$; `/* the other caches connected to` v `(other than the previous cache` v'`). */`
if $P = \emptyset$ **then**
  return B ; `/*` v `is the root, so forward to the backing store */`
end
return CHOOSENEXT(P) ; `/* Choose the next cache from` P `*/`

Algorithm 4's purpose is to point to the next cache to which a given request should be forwarded to. Given the network and the current cache vertex, it finds all possible candidates (i.e. all those that are connected to the current vertex but not the one that originally forwarded the request to v). If this set of candidates is empty, then that means the request has fully missed and so it is returned to B. Else, using some *deterministic* decision rule, CHOOSENEXT, return the next-hop vertex. It should be noted that herein, CHOOSENEXT is not explicitly defined as the simulation by default chose the 'first' vertex in the set. Furthermore, in both line and tree networks, there is never a choice for the next hop, so no other decision rule was required.

Algorithm 5: PATH-TO

Data: The network N; a source vertex src; a destination dst (default b).
Result: The sequence of vertices traversed from src to dst.
$p \leftarrow$ (src);
$v \leftarrow$ src;
while $v \neq$ dst **do**
  $v' \leftarrow$ NEXT-HOP(v);
  if $v' = \perp$ **then break**;
  $p \leftarrow p \oplus (v')$; `/*` $\oplus$ `is sequence concatenation */`
  $v \leftarrow v'$;
end
return p

Algorithm 5 rebuilds the path taken by any request, given their starting cache (the ingress), and their destination (either B or whichever cache at which the request hit). Since everything in the framework is deterministic, this can eb rebuilt at the end of the request's 'journey' through the network in order to compute the latency incurred by the request.

Algorithm 6: FORWARD

Data: A request $r = (i, t_r)$; the vertex v at which it has arrived.
Result: A status $s \in \{\text{MISS}, \text{DH}, \text{HIT}, b\}$ and the next vertex v', or $\perp$ if the request halts here.

if $v = b$ **then**
  RECV-FETCH(r) ; `/* the backing store begins the fetch */`
  return $(b, \perp)$
end
if $\exists \tau : (i, \tau) \in C^{(v)}$ **then**
  $C^{(v)} \leftarrow \text{REPLACE}\big(C^{(v)},\ (i, \tau),\ (i, t)\big)$; `/* hit: restamp the entry */`
  return $(\text{HIT}, \perp)$
end
if $i \in Q_F(v)$ **then**
  $Q_R(v) \leftarrow Q_R(v) \oplus r$; `/* delayed hit: park behind the in-flight fetch */`
  return $(\text{DH}, \perp)$
end
$Q_F(v) \leftarrow Q_F(v) \oplus i$; `/* miss: mark in flight and propagate upstream */`
return $(\text{MISS},\ \text{NEXT-HOP}(v))$

However, in order to prevent scope-creep and ensure the simulations were completed in time, the the implemented versions of the adversarial sequences and $\mathcal{O}$ instances were written specifically *for* line-networks of an arbitrary size.

6.1.3 Implementation of $\mathcal{O}$

Defining a general-case $\mathcal{O}$ offline algorithm for DH-aware cache networks is outside of the scope of this project. Here, only specific scenarios and the way $\mathcal{O}$ behaves is defined and is analysed. Thus, in this project, there exists no general-case algorithm which can then be implemented in this framework. Therefore, only the specific $\mathcal{O}$ instances that exists in the scenarios provided in Section 5 are implemented. An example instance is given below, the exact instance used for $\{\rho_n\}$ sequence-family $\mathcal{O}_n$:

6.1.4 Dynamic Creation of Sequences

The sequences constructed in Section 5 are, largely, defined by the *algorithms* that constructed them, in order to maintain generality. Thus, instead of being individual sequences per se, they are families of sequences whose specific items can change depending on the situation. Therefore, in the simulation, the sequences cannot, typically, be created prior to any running of the simulation. Instead, there must be one pass through of the simulation in order to create the full sequence, to determine what requests must

be made in response to the policy being used.

6.2 Simulation Specifics

As mentioned previously, there were, in total, three simulations run. Each one traced a specific sequence (one of ρ_{MW}, ρ_A, or $\rho_n \in \{\rho_n\}$) and computed the competitive ratio for various growing values of $|V|$, k, Z_A, and, if applicable, N_μ. This 'sweep' was done in a purely inefficient, brute force fashion: iterating over all three (or four) variables using nested-for loops, and running the specified simulation for each one. The chosen values were as follows:

- k: $\{1, 2, 4, 8, 16, 32, 64, 128\}$
- $|V|$: $\{2, 4, 8, 16, 32, 64, 128\}$
- Z_A: $\{1, 2, 4, 8, 16, 32\}$

Furthermore, $|\mathcal{I}|$ was chosen to be $|\mathcal{I}| = |V|k + 1$, in order to ensure the constraint $|V|k < |\mathcal{I}|$ always holds, maintaining non-triviality. Finally, as per Section 5, $TTL = 6$ for all simulations.

6.2.1 Simulation Flowchart

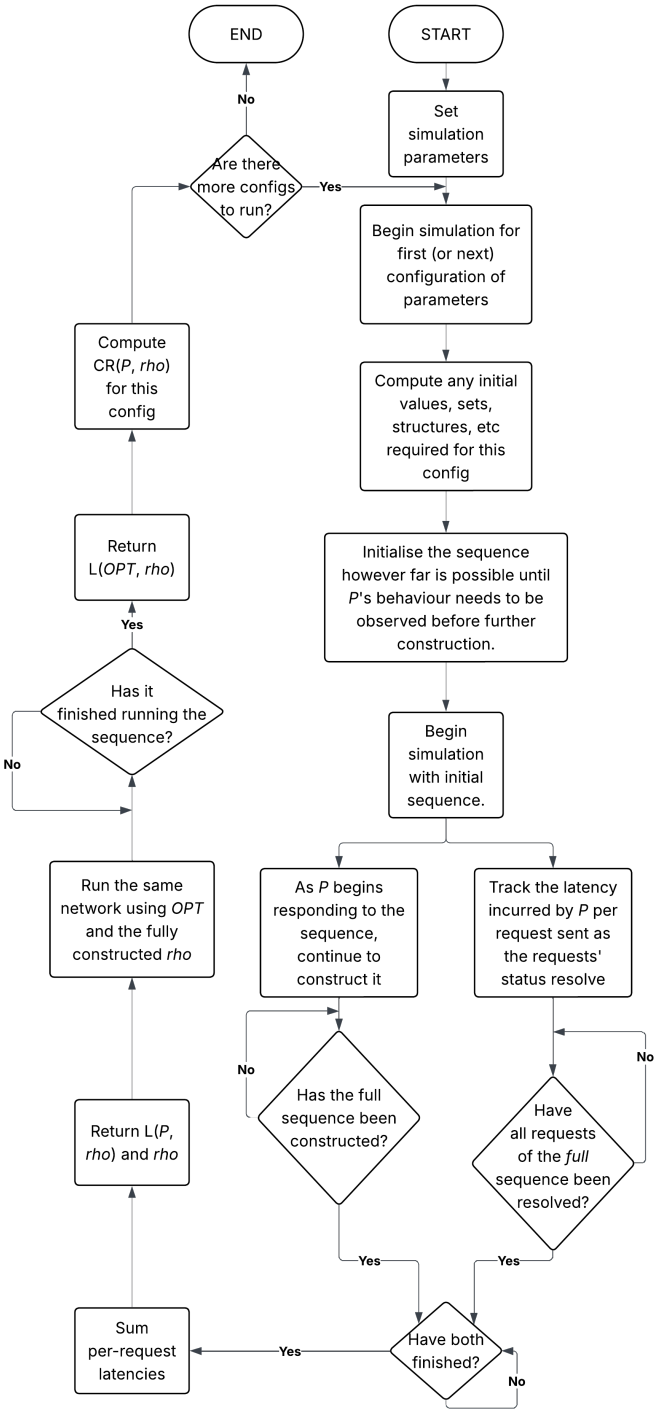


Figure 6.2: A flowchart showing the process for each simulation herein.

From Figure 6.2, the process for obtaining the results in each simulation can be observed. In the code, this process maps `comp simp.py, simulateBuild.py,` and `opts.py`. First, the initial simulation parameters are set. These are values that exist in the general framework: Z_A, k, $|V|$, $|\mathcal{I}|$, etc. The values chosen, specifically, are *ranges* to iterate over. Tracking the CR's growth with relevant variables requires the variables to grow incredibly large, thus feasibly large ranges are used. The flowchart represents the process for *one* configuration of the core three variables, for *one* sequence.

After the first configuration has been computed, first any follow-up parameters, structures, or variables are computed or constructed. For example, this will be Z_B and k_T, or the set W if the simulation is for ρ_A (Definition 5.3.1, Section 5.3). After this, the simulator will construct the *initial* part of the sequence however far it can until it requires the policies' behaviour in order to proceed further.

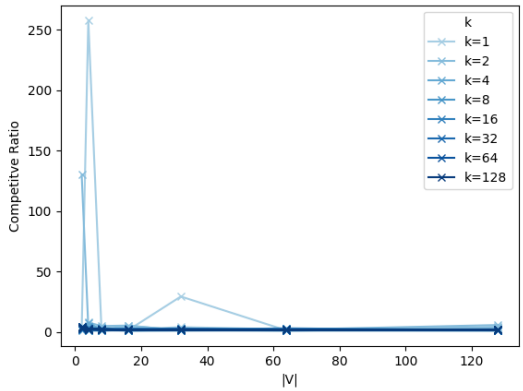
Following this, the *dynamic creation of sequences* subprocess is executed, as explained briefly in Section 6.1.4. The process will first proceed with simulating the initial, already-built portion of the sequence on the online policy $\mathcal{P}$. It will, at the same time, record the latency of each initial request. After this point, the flowchart splits into two portions. It should be clearly noted that this does **not** represent a decision of any sort, but it is a design choice to convey the fact that these processes are happening 'at the same time' instead of one after the other. This is because, if the simulation *has* to run in order to fully construct the sequence, then it is more efficient to measure the latency incurred by the sequence at the same time. Once the sequence finishes and all latencies have been recorded, $L(\mathcal{P}, \rho)$ is computed.

Then, the now-fully-built sequence is run on the same network but this time with the offline policy $\mathcal{O}$. After the full sequence runs, again, the latency is summed and $L(\mathcal{O}, \rho)$ is returned. Finally, $\text{CR}(\mathcal{P}, \rho) = \frac{L(\mathcal{P},\rho)}{\mathcal{O},\rho)}$. This is the final step for the configuration. After this, the same process for the next configuration is executed, until there are no outstanding configurations left.

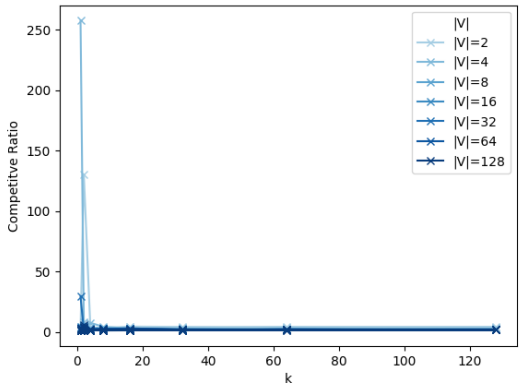
6.3 Simulation of ρ_{MW} on Line Networks

The purpose of this simulation was to empirically confirm that ρ_{MW} as the input sequence causes the system's CR to grow independently of $|V|$, Z_A, and k. In other words, the competitive ratio should be the same no matter what value any of these variables hold, i.e. it should grow in $\Omega(1)$. Figure 6.3 shows exactly this. In both figures, the competitive ratio stays relatively constant (at a near-zero value) while either variable grows large.

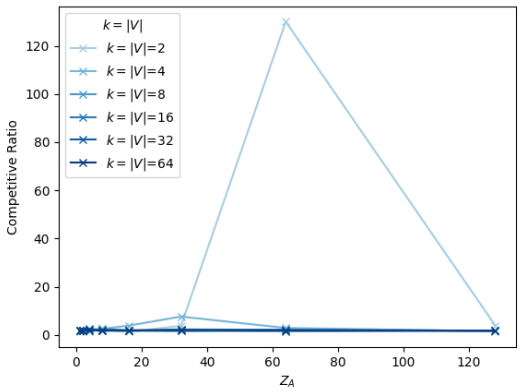
However, some volatility and anomalous results can be observed. Particularly at $|V| =\sim 38$ (Figure 6.3b) and at $k =\sim 2$ (Figure 6.3a). Here, the competitive ratio deviates from others at a similar x-axis value.



(a) ρ_{MW} while fixing k



(b) ρ_{MW} while fixing $|V|$



(c) ρ_{MW} while fixing Z_A

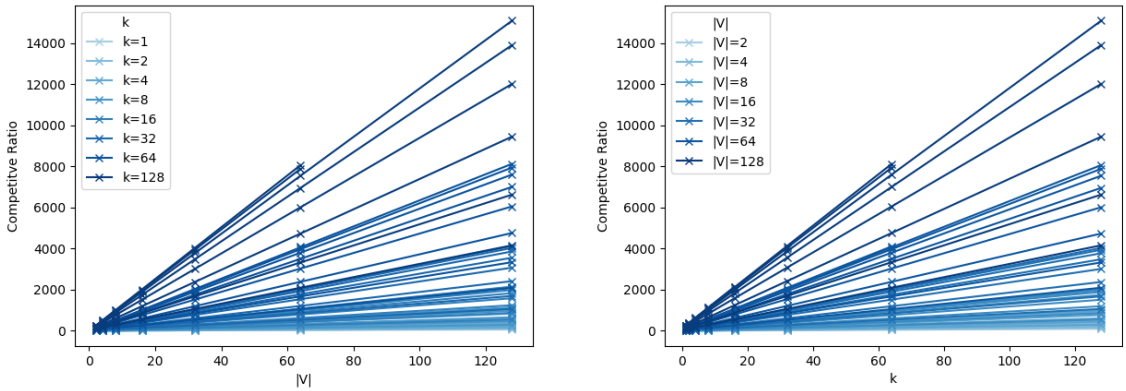
Figure 6.3: ρ_{MW} comparison for different fixed parameters

6.4 Simulation of ρ_A on Line Networks

This simulation shows that, in contrast to the previous one, ρ_A's competitive ratio *does* grow with network-specific parameters. Figure 6.4 shows this perfectly: Both sub-figures show the competitive ratio growing proportionally to each variable, whereas it stays constant respective to Z_A. This is exactly what is shown by Theorem 5.3.1 ($\Omega(k|V|)$).

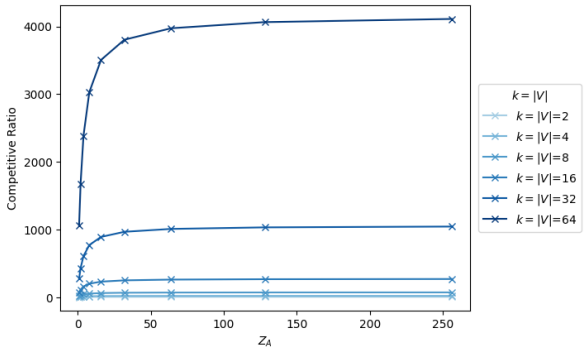
It should however be noted that in Figure 6.4c, there is a sharp initial growth in the competitive ratio, for relatively small increases in Z_A. This can be attributed to initial volatility often observed in systems at small values for variables. At such values, the behaviour of the system may still be dominated by hardware- or implementation-related issues, resulting in initial curves that appear to contribute to the overall trend of the data when they do not. In fact, Figure 6.4c, for all large values of Z_A, shows

exactly what the theorem proposed.



(a) ρ_{A} while fixing $|V|$

(b) ρ_{A} while fixing k



(c) ρ_{A} while fixing Z_A

Figure 6.4: ρ_{A} comparison for different fixed parameters

6.5 Simulation of $\{\rho_n\}$

This final simulation empirically validates the results obtained in Section 5.4, which showed that the LCE policy is uncompetitive. The simulation tested multiple growing values of k, $|V|$, and Z_A against very large and growing values of N_μ (the length of the 'deep-hits' portion of the sequence ϱ). Figure 6.5 shows the result of this simulation: the competitive ratio grows proportionally to the number of requests being made for the targetted item g, whilst the bands for each $k = |V|$ value remain close together, indicating that the competitive ratio is not changing significantly with network parameters.

- $k = |V|$: $\{2, 3, 4, 6, 8, 12, 16\}$
- Z_A: $\{1, 2, 4, 8, 12, 16, 24, 32\}$

- N_μ: $\{1024, 1448, 2048, 2896, 4096, 5793, 8192, 11585,$
  $16384, 23170, 32768, 46341, 65536, 92682, 131072\}$

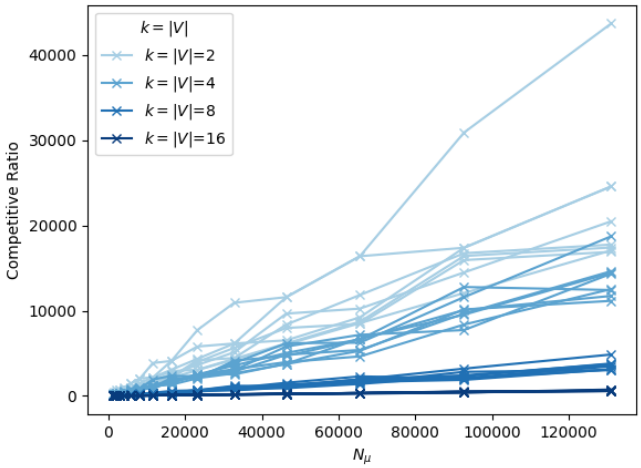


Figure 6.5: Enter Caption

6.6 Further Verification of Theory Using Cumulative Latency Graphs

The figures above validate the theoretical results outlined earlier. For further clarification, however, the following cumulative latency figures are provided below. These figures show, for an arbitrary line-network configuration, how the cumulative latency incurred by LCE and $\mathcal{O}$ diverge as the sequence is driven through the network, thereby showing clearly per-sequence competitive ratios.

Figure 6.6 shows both $L(\text{LCE})$ and $L(\mathcal{O})$ scaling almost in-parallel to one another. In other words, LCE's latency grows in-line with $\mathcal{O}$, meaning the ratio between them, as time progresses, remains more-or-less the same. Thus, implying that $\text{CR}(\text{LCE}, \rho_{\text{MW}}) = \Omega(1)$.

On the other hand, Figure 6.7 shows that, while $L(\text{LCE})$ continues to grow at a linear rate with ρ_A, $L(\mathcal{O})$ remains, essentially, constant throughout. From Section 5.3, it is known that $\mathcal{O}$ incurs a latency of no more than $Z_B + Z_B \cdot Z_A$, which Figure 6.7 now confirms.

For both Figures 6.6 and 6.7, note how both policies grow in a 'steps'-trend: this is the result of bursty requests causing policies to incur latency in a short period of time, followed by a period of time of low-latency activity (typically due to no requests).

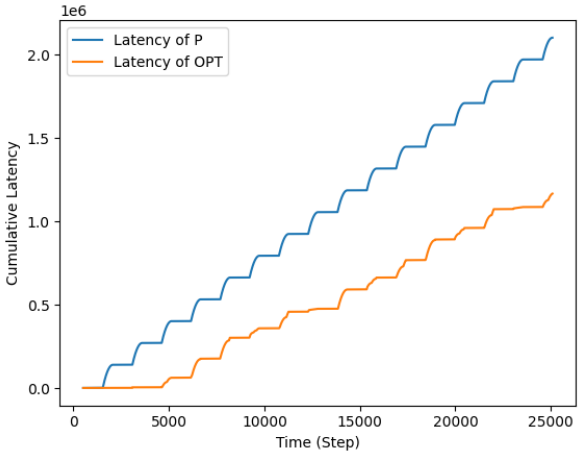


Figure 6.6: Cumulative Latency of any $(k, |V|, Z_A)$-network configuration using ρ_{MW}

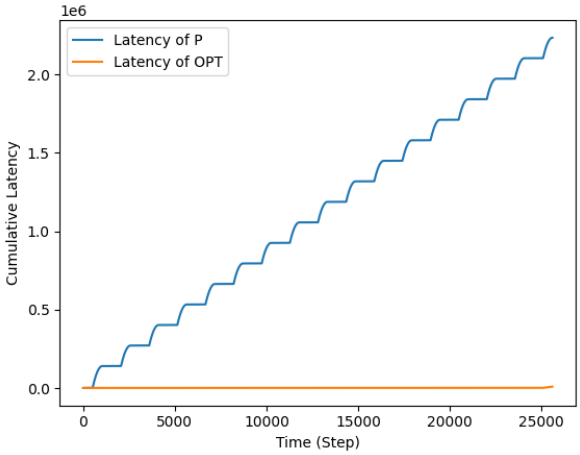


Figure 6.7: Cumulative Latency of any $(k, |V|, Z_A)$-network configuration using ρ_A

Figure 6.8 shows the cumulative latency graph for the $\{\rho_n\}$ sequence-family. An important point to make about this specifically is that, unlike the others, the specific pattern of the plot changes depending on the network configuration. As stated in Section 5.4.1, the initial part of the sequence φ, continuously makes requests in such a way that it manipulates LCE to keep some targetted item g only at a deep cache vertex. This requires making multiple strategic requests to the network to force the network to evict g from every other shallower cache vertex. Such a series of request depends heavily on both k and $|V|$ intuitively, and so the point at which φ 'ends', and the next subsequence μ begins, can change drastically across configurations.

As such, in Figure 6.8, three points of significance have been labelled: The latency incurred by the request at $t = Z_B$ (this would have been for g), the latency incurred when g was requested for the first time *after* the very first request, and the last latency

recorded by $\mathcal{O}$. The middle point is significant because subsequence μ will only ever request item g in order to force $L(\text{LCE})$ to grow quickly. Furthermore, φ only requests g once, as part of its strategy, as the very first request. So, the next time g is requested is when μ begins. This is also the point at which $\mathcal{O}$ *stops* incurring any latency. Thus, the first half of the figure shows that both $\mathcal{O}$ and LCE incur the same or similar latencies, both of which grow, until $t = 19\ 275$. After this point, LCE experiences a drastic increase in latency as μ continuously requests g (which hits deeply with LCE). Meanwhile, $\mathcal{O}$ placed g at $C^{(0)}$ at the first chance it could, so it incurs no more latency.

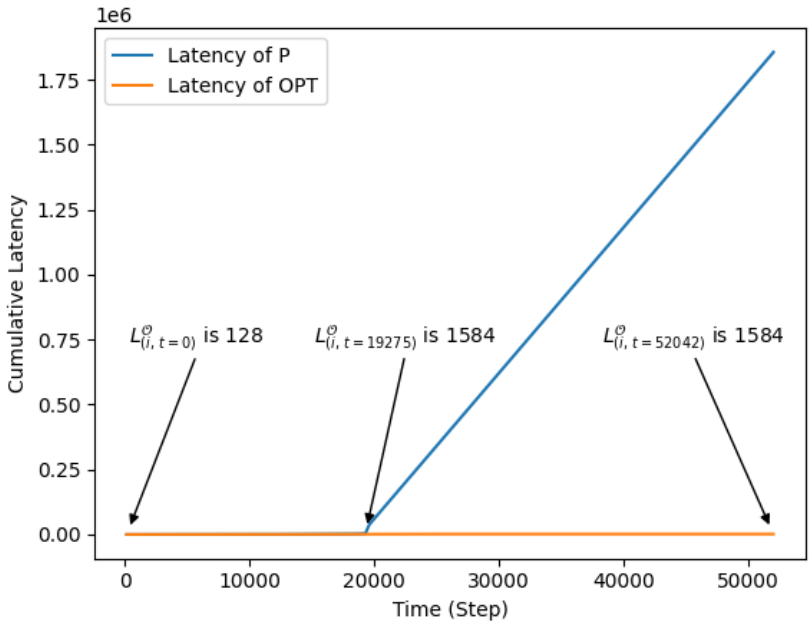


Figure 6.8: Cumulative Latency of any $(N_\mu = 32\ 768, k = 8, |V| = 8, Z_A = 16)$-network configuration using $\{\rho_n\}$

Thus, the two cumulative latencies diverge quickly. Since $\mathcal{O}$ incurs no more latency, it is completely invariant to the number of requests N_μ that μ makes. However, LCE will only incur the additional μ-latency for however long μ continues to request g for. Thus, $L(\text{LCE})$ is *completely* dependent on N_μ. Therefore, this contributes to the evidence, and supports the theoretical findings, that LCE is unbounded above and is therefore uncompetitive.

In this chapter, the framework that was developed alongside the theory was introduced. Key parts, components, and implementation choices for the framework were further discussed in detail. Before then reviewing the results from the simulations, the actual process and methodology for carrying them out was explained and examined. All results gathered from all three simulations do indeed empirically validate their theoretical counterparts:

- Under ρ_{MW}, the competitive ratio is invariant to all three network parameters, as shown by the median ratio 1.92 across all 336 configurations.
- For ρ_A, the growth trend of the ratio in all three figures tracks with the obtained bound of $\Omega(k|V|)$: the ratio grows linearly with k and $|V|$ when the other is 'fixed', whilst the ratio is also shown to be invariant to Z_A's growth.
- $\{\rho_n\}$ (ϱ): From the provided data it can be seen that there is a real relationship between the growth of the competitive ratio and the length of the *second* portion of the sequence (μ). From the graph provided, this trend appears to be linear which validates the theoretical findings: the sequence length is the only variable that dominates as it grows large and so it means that LCE is uncompetitive.

Chapter 7

Exploration of Tree Topologies

In this chapter, an extension to tree networks is explored. First, such a network topology is formally defined using the existing model(s) in Chapter 4. Then, building on the model, avenues of further research are outlined informally, with justifications regarding their suitability as natural extensions of the current work.

7.1 Formal Model

First, the formal model for cache networks, as outlined in Chapter 4, is restricted to tree topologies, as was done with line topology in Section 4.2.2.

Definition 7.1.1 (Cache Tree Networks): A cache network $\mathcal{N}$, as in Definition 4.2.1, is a *tree network* (tree topology) iff:

1. There is exactly one vertex, the *root* r, that is connected to B;
2. The underlying graph is connected and acyclic (i.e. a tree), rooted at r;
3. Every leaf vertex is an ingress vertex; and
4. Every maximal path from an ingress leaf to r is a cache line-network (Definition 4.2.4), with r as its unique B-connected vertex.

Remark (Tree and Line Networks): From Definition 7.1.1, a tree network decomposes into its root-to-leaf line-networks, which share their B-side sub-paths and diverge at branch vertices.

While line-networks provide a foundational model for caching, real-world infrastructures often involve branching paths. These are defined these as *tree networks* in Definition 7.1.1. The significance of this structure is captured in the above remark:

despite the complexity of the tree, it can be decomposed into multiple, brancing root-to-leaf line-networks. This decomposition is visually represented in Figure 7.1.

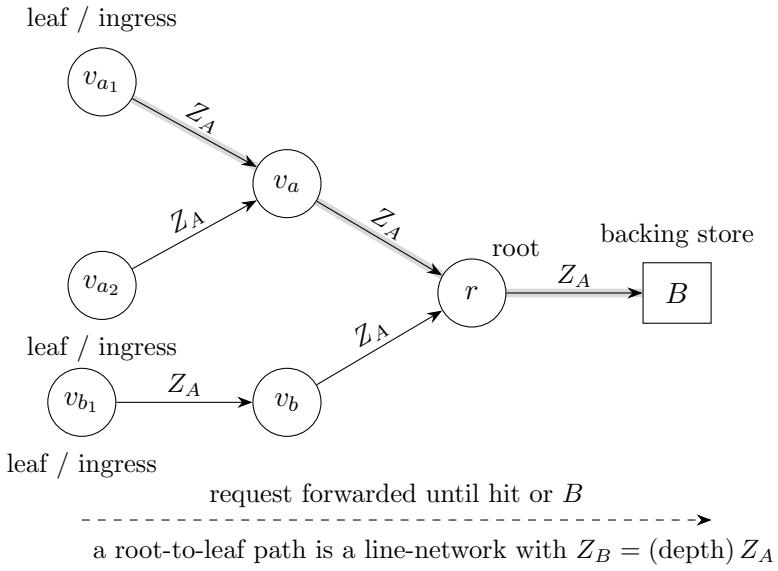


Figure 7.1: A tree network of caches

7.2 Multiple Requests

From Figure 7.1, there is a key difference between the way caching behaves on line networks versus tree networks: Two caches (e.g. v_a and v_b) may forward requests to another root cache (e.g. r) at the same time, whereas caches in line-networks dont get a choice of *where* to forward to. The root node then has to rely on some decision rule that dictates how to prioritise the incoming requests. An alternative modelling approach is to utilise superposition, which has traditionally been applied in continuous-time models. Berger et al. in 2014 used Markov Arrival Processes with superposition to resolve this [13].

7.3 Considerations for $\mathcal{O}$

In line-networks, the specific $\mathcal{O}$ algorithm ($\mathcal{O}_{\text{LINE}}$) only has to make optimal decisions regarding depth of placement. However, a possible $\mathcal{O}$ algorithm for tree networks ($\mathcal{O}_{\text{TREE}}$) must also consider 'exclusivity'. That is, if item i is being requested multiple times at two separate branches in the future (e.g. at v_{a_1} and at v_{b_1}), but with no opportunity to change placement in the network between them, then where is the most optimal placement for i in the network? $\mathcal{O}_{\text{TREE}}$ can:

1. Place the item at some point in *one* of the branches;

2. Place the item in *both* branches (duplication, like LCE); or

3. Place the item at the root cache of the two branches.

Option one means that $\mathcal{O}_{\text{TREE}}$ must suffer cache-misses on one set of requests, option two means that $\mathcal{O}_{\text{TREE}}$ has to make do with redundancy, and possibly waste a slot that could have been used later. Finally, option three means that it will have to suffer multiple deep hits, at a possibly high latency cost. Such complexity in placement decisions does not exist for $\mathcal{O}_{\text{LINE}}$.

7.4 LCE may also be Uncompetitive on Trees

One possible avenue of investigation that can branch from Theorem 5.4.1 is the possibility of LCE also being uncompetitive on tree topologies. The family of sequences that prove LCE is unbounded above would isolate items that are stored only on one path from leaf to B, thereby, effectively, reducing the tree to a line. Then, a similar composition of requests could be made to force a specific item to be stored somewhere, deeply, in that path. As with the line network, the adversary would then continuously request this item.

In this chapter, the extension of the current work to tree topologies has been explored. Specific ideas and paths of investigation and research have been outlined that act as natural next steps in DH-aware competitive analysis for caching networks. In the next chapter, the work as a whole will be summarised and evaluated, and the management of the project will be appraised.

Chapter 8

Conclusion and Evaluation

8.1 Project Management

Throughout the time allocated to the project, regular meetings were held with the supervisor to review progress and discuss any issues that arose. The subsequent advice given was taken onboard and acted upon. This meant that there was focused effort at each stage. Progress broadly followed the original timetable set out in the interim report (Appendix **??**). The lower-bound analysis was completed on-schedule. This then led to the upper-bound work described in Section 5.4. The simulation framework was developed in parallel to the theoretical analysis, as well.

8.1.1 Unforeseen Problems

Over the course of the project, there were two notable deviations from the original plan. The first was with the implementation of $\mathcal{O}$ during framework development. The original plan discussed the development of a fully generalised framework, able to compute the competitive ratio for any caching topology and any CPP. This would require $\mathcal{O}$ to be implemented in a general sense. Such an algorithm is notoriously difficult to not only identify, but also to prove and then develop. As noted in Section 5, this is outside of the scope of this project. Thus, the framework is unable to compute the competitive ratio for any sequence on a line-network. It is only able to do so for the adversarial sequences provided herein. This is because the specific reduced instances of $\mathcal{O}$ were implemented directly. This still permits empirical validation of the theoretical results, though.

The second deviation is regarding the upper-bound. The original plan stated that the objective was to establish a matching upper-bound for LCE, which would thereby provide a tight guarantee. As stated in Sections **??** 5.4, it was eventually proven that the LCE policy is unbounded above. The intended deliverable therefore changed, but the core intent of the objective was still met.

8.1.2 Appraisal and Reflection

I drew an initial project timetable (see appendix so and so). Progress has been kept in line with this despite competing academic demands. This because these demands were accounted for already in my project plan and I ensured that I worked consistently throughout the academic year.

Throughout this project, I improved my research methodology skills. For example, with approaching the analysis and lower-bounding efficiently. Initially, the approach taken to find the adversarial sequence was that of 'trial and error': observing what works and doesn't work with one sequence, and iterating it in an attempt to improve it. I further utilised techniques found and developed in existing literature to aid the 'development' process of the sequence(s). After I had gained the necessary intuition for sequence construction, I was able to informally reason fully about sequences and determine, much more efficiently, what worked well and what did not. I then developed the required skills to formalise such reasoning and to write the required proofs.

The lower-bound analysis was thus completed on schedule, adequately setting up the start of the upper-bound analysis. The upper-bound analysis began, in earnest, with attempting to apply existing conventional techniques such as phase partitioning. However, such techniques failed to work as they relied on properties that were simply violated in the network model and were non-recoverable. As such, I turned then to other techniques such as Primal-Dual linear programming. However, after these also did not yield many meaningful results, I considered the possibility that LCE was uncompetitive. A framework for simulation and a simulator tool was developed to validate the theoretical results.

Throughout this project, I communicated and interacted consistently with my supervisor. To support these regular meetings, I got into the habit of keeping a written record right from the start. This helped me stay on track.

8.1.3 AI Usage

Large Language Models (specifically Claude Opus 4.8 and 5) were used to challenge and scrutinise ideas and analyses. They were primarily used as soundboards when I proposed ideas regarding sequence construction and when I reasoned about the network's behaviour. For instance, I asked it to challenge arguments and attempt to find holes my logic in such a way that it acted as an informal viva. This was used as opposed to generating raw results, ideas, and proofs from it. It was explicitly told not to provide solutions, major ideas, or proofs. All theoretical contributions, constructions, and proofs are my own.

8.2 Conclusion

This project's original core objective was to provide a tight guarantee on the behaviour of a specific CPP on a line network. This was to be characterised by network-specific parameters such as number of caches $|V|$ and the inter-cache delay parameter Z_A. This work was motivated by the ever growing presence of caches as or in networked systems, and the need to understand the possibilities and limitations of such systems.

A non-trivial candidate for the lower-bound was provided, that being $\Omega(k|V|)$. This indicates that the lower-bound grows as the number of caches $|V|$ and the size of each cache k grows. This is an improvement on the earlier result, which was a trivial $\Omega(1)$, indicating that the lower-bound is independent of network-specific parameters. Such a result was obtained by using methods that have recently been employed on single-cache systems for a similar purpose.

This project was unable to identify an upper-bound for the LCE policy. The cause of this issue, actually, was due to the fact that LCE is unbounded from above. In other words, this report proves LCE to be *uncompetitive*. This was done by finding a family of sequences in which LCE's competitive ratio grows independently of the network's parameters. Instead, this family shows the CR growing arbitrarily with the number of requests made. Thus, LCE is not competitive.

All three of these central results, the two lower-bounds and the uncompetitive nature of LCE, were then empirically validated using a custom-made simulation framework. The results of such simulations were then analysed and evaluated by comparing the empirical results with the theoretical findings. All simulation results did, indeed, validate the theory.

Finally, this report explores possible extensions of this work. Specifically, it explores the possibility of extending this work with tree topology networks as the next step.

8.3 Evaluation

This project, and report, can be evaluated by comparing the final obtained results with the prescribed objectives in Chapter 2. Objectives 1, 2, 3, 4 and 5 were completed fully in the expected manner: The caching model was formally defined, and a valid, non-trivial lower-bound was established. Then, it was proven that the LCE policy cannot be bounded above. This objective can be marked as met: as discussed in Section 5.4, there is no upper-bound to find, which means that there cannot be a tight-guarantee, and that this result is the next best achievement. Thus, proving uncompetitiveness suffices for fulfilment of the objective. Further, these bounds were validated by the simulations that were carried out in Chapter 6.

Objective 1 required a full DH-aware caching model to be defined, along with the behaviour of the LCE CPP. This was fulfilled in Section 4, which outlines a fully formalised caching model for single-cache systems, delayed-hits, and caching networks.

Along with this, it provides the formal algorithms for both TTL and LCE.

Criteria of objective 2 involved establishing an initial lower-bound using the Manohar-Williams adversarial sequence, and using any issues present in that (when used in a line network) to create a new adversarial sequence to establish a 'better' lower-bound. This was fully achieved. As stated previously, the initial lower-bound was trivial and the issue identified was that the adversary was aggregating the cache network as one, large cache. It did not fully take into account the fact that even the optimal algorithm has to incur latency on some hits, and so both latencies grow at the same speed, relatively. This knowledge was then used to design a sequence that successfully charged LCE, whilst allowing $\mathcal{O}$ to avoid incurring high latency.

Objective 3 states that a corresponding upper-bound should be found for LCE, thereby showing tightness (by matching the two bounds). After various attempts at trying to find an upper-bound of any kind (not just a matching one) failed to provide one, the possibility of uncompetitiveness was considered. This was then proven. Therefore, although not achieving the exact requirements of objective 3, it has been proven that it is not possible to achieve it. The LCE policy has no upper-bound of any kind for the network-specific parameters that are being investigated here. Thus, objective 3 has been met as a definitive statement on the upper-bound, specifically the lack of it, has been made.

Objectives 4 and 5 were also completely fulfilled. The provided framework is able to accept a line-network of any size and is able to apply policies, which can be implemented in a relatively easy way via helper functions. The framework also allows for the variation of certain parameters of the simulations. This framework was then used to successfully validate the theoretical findings.

Overall, this project and report has been successful in fulfilling its prescribed objectives. All objectives were completed.

8.4 Future Work

In this section future work, which is outside of the scope of this project, will be discussed as next steps that would fill current gaps in caching research.

8.4.1 Formal Definition for $\mathcal{O}$

As stated previously, this work relies on *reductions* of the $\mathcal{O}$ policy. That is, this report identifies and applies $\mathcal{O}$ in only a very small subset of scenarios such that the identification of $\mathcal{O}$'s behaviour becomes a more tractable problem. This had to be done because, to date, there is no work proving the optimal, offline cache placement policy for networks. Thus, before proceeding further, such a generalised algorithm should be defined and proven. This algorithm should hold for all networks, of any topology in the

most ideal case. A sufficient roadmap would be first identifying $\mathcal{O}$ for specific toplogies, and then attempting to unify these into a generalised algorithm.

8.4.2 Generalisation to Other or All Topologies

This work focuses solely on the competitive analysis on *line*-networks, so a natural extension would be an exploration into (i) other topologies; and (ii) arbitrary, general topologies. This has already somewhat been explored in Chapters 7 with tree topologies. However, this is only an initial exploration and discussion, and not a complete analysis. Subsequent research could first explore other popular network topologies, such as star or ring topologies, and eventually provide then a unifying analysis for a network with an arbitrary topology.

8.4.3 Extension to Other CPPs, or Exploration into CPP-Invariant Bounds

This project specifically uses the Leave-a-Copy-Everywhere content placement policy. Future work should explore competitive analysis for other CPPs and eviction policy-CPP combinations. The end goal should, therefore, be a competitive analysis that is fully generalised. That is, it is eviction policy- and content placement policy-invariant. However, the next immediate steps should be to extend this work for other popular CPPs, such as Promote and Demote under a DH-aware context.

Appendix A

Timetable

A.1 Overall Initial Timetable

Original timetable for the project, made in February/March 2026.

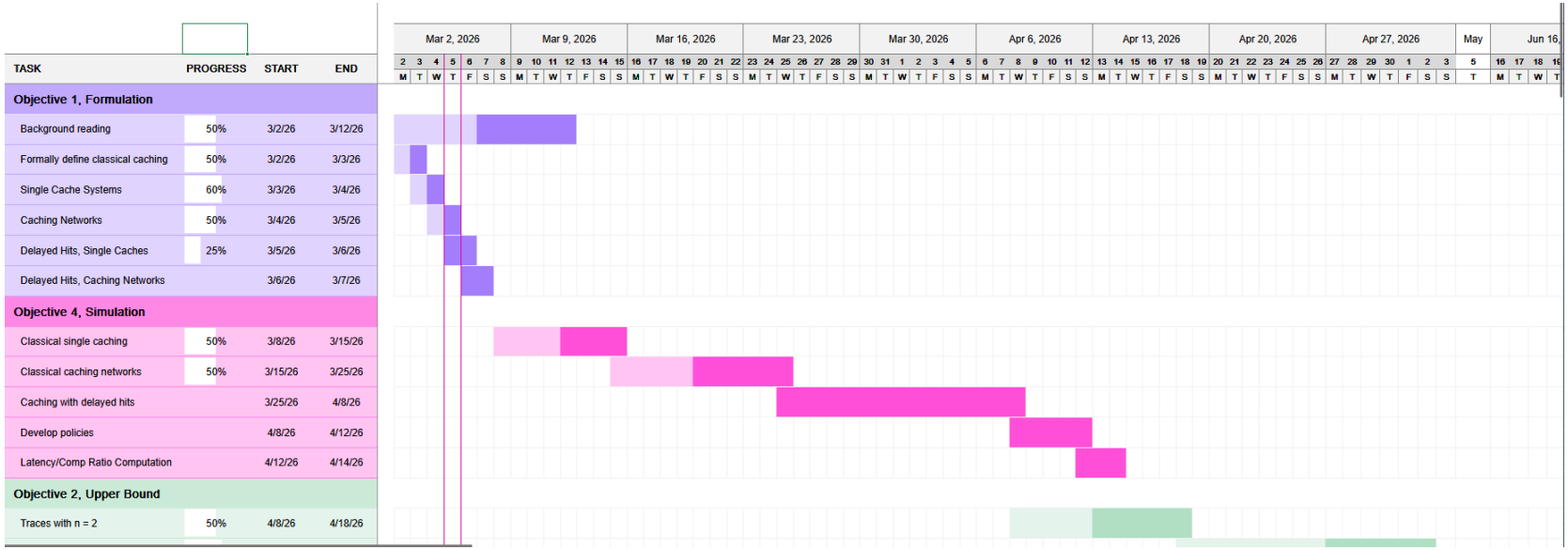


Figure A.1: Gantt Chart March - April

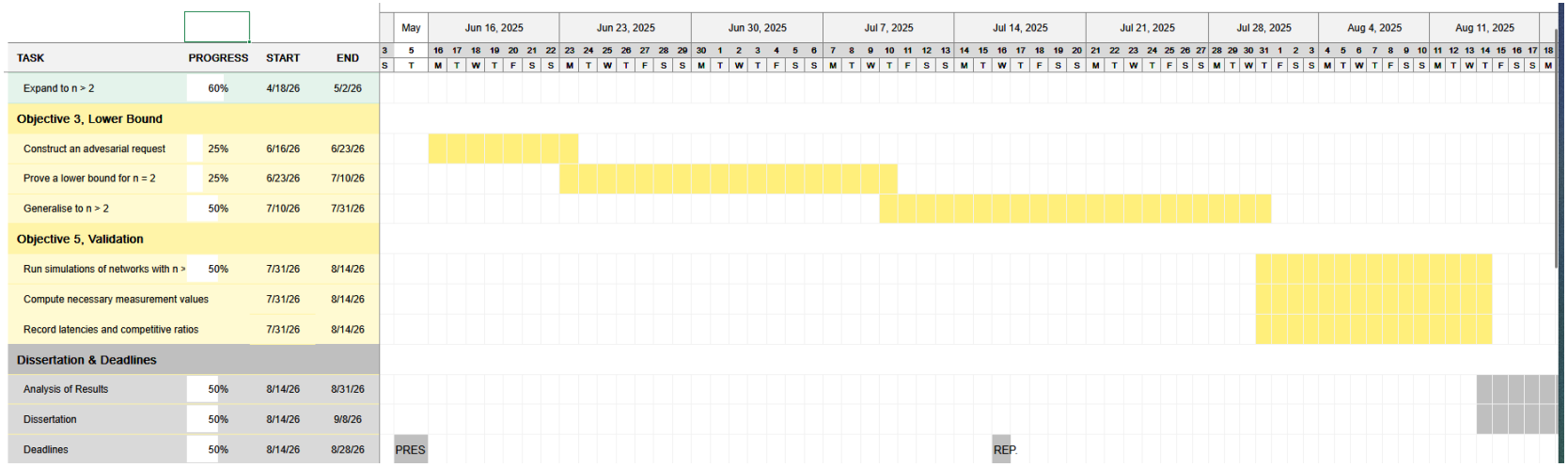


Figure A.2: Gantt Chart June - August

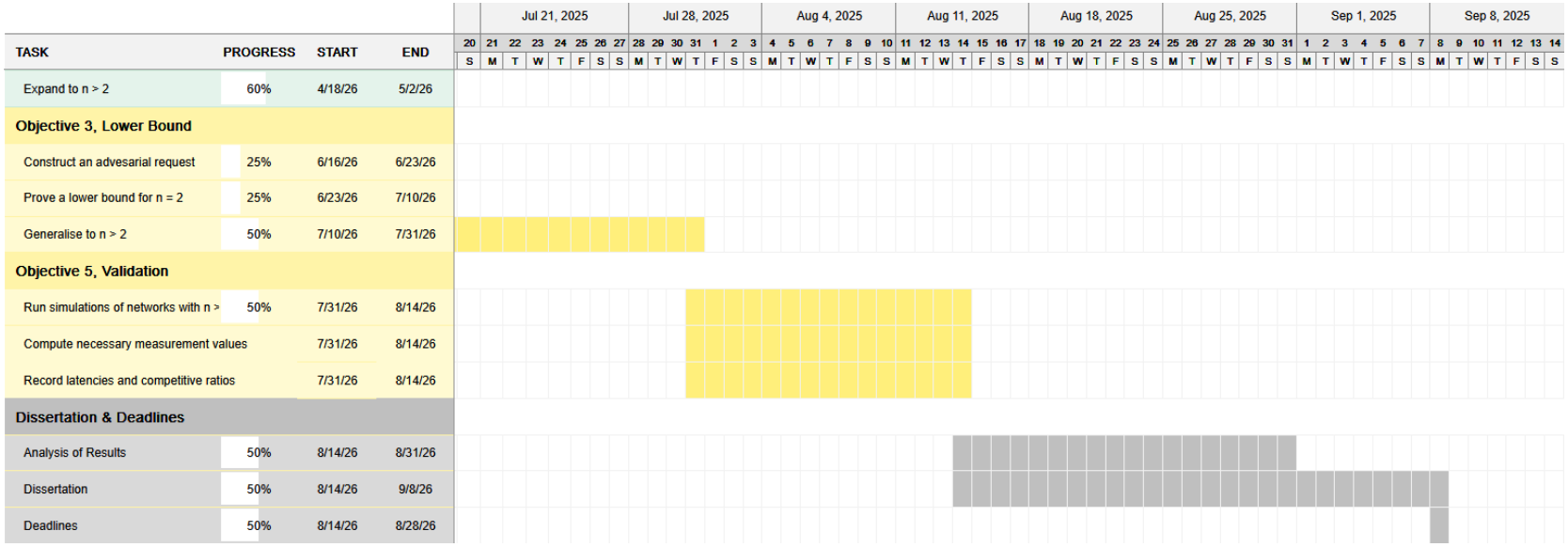


Figure A.3: Gantt Chart August - September

A.2 Post-Interim Timetable

Deliverable	**Window**
LCE upper bound	3 Weeks
Finalise simulation & record findings	2 Weeks
Consolidation, write-up, and revision (parallel)	6 Weeks

Bibliography

[1] Cloudflare, "What is an edge server?" https://www.cloudflare.com/en-gb/learning/cdn/glossary/edge-server/, n.d., accessed: 23 February 2026.

[2] B. Jiang and C. Ma, "Modeling and optimizing latency for delayed hit caching with stochastic miss latency," *arXiv preprint arXiv:2505.15531*, May 2025. [Online]. Available: https://arxiv.org/abs/2505.15531

[3] B. Wu, W. Bao, and B. B. Zhou, "Competitive analysis of online elastic caching of transient data in multi-tiered content delivery network," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 12, pp. 2449–2462, 2024.

[4] N. Atre, J. Sherry, W. Wang, and D. S. Berger, "Caching with delayed hits," in *Proceedings of the ACM SIGCOMM*. ACM, 2020, pp. 495–510.

[5] A. Jain, C. Lin, N. Jerger, and M. Martonosi, *Cache Replacement Policies*, ser. Synthesis Lectures on Computer Architecture. Morgan & Claypool Publishers, 2019. [Online]. Available: https://books.google.co.uk/books?id=QRieDwAAQBAJ

[6] S. Ioannidis and E. Yeh, " Adaptive Caching Networks With Optimality Guarantees ," *IEEE/ACM Transactions on Networking*, vol. 26, no. 02, pp. 737–750, Mar. 2018. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TNET.2018.2793581

[7] A. Ioannou and S. Weber, "A survey of caching policies and forwarding mechanisms in information-centric networking," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 4, pp. 2847–2886, 2016.

[8] G. Zhang, Y. Li, and T. Lin, "Caching in information centric networking: A survey," *Computer Networks*, vol. 57, no. 16, pp. 3128–3141, 2013, information Centric Networking. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128613002235

[9] P. Manohar and J. Williams, "Lower bounds for caching with delayed hits," *arXiv preprint arXiv:2006.00376*, 2020. [Online]. Available: https://arxiv.org/abs/2006.00376

[10] T. Kilburn, D. B. G. Edwards, M. J. Lanigan, and F. H. Sumner, "One-level storage system," *IRE Transactions on Electronic Computers*, vol. EC-11, no. 2, pp. 223–235, 1962.

[11] L. A. Belady, "A study of replacement algorithms for virtual-storage computer," *IBM Syst. J.*, vol. 5, pp. 78–101, 1966. [Online]. Available: https://api.semanticscholar.org/CorpusID:8344798

[12] Z. Li, G. Simon, and A. Gravey, "Caching policies for in-network caching," 07 2012.

[13] D. S. Berger, P. Gland, S. Singla, and F. Ciucu, "Exact analysis of ttl cache networks," *Performance Evaluation*, vol. 92, pp. 79–91, 2014.

[14] T. Wong and J. Wilkes, "My cache or yours? making storage more exclusive," in *2002 USENIX Annual Technical Conference (USENIX ATC 02)*. Monterey, CA: USENIX Association, Jun. 2002. [Online]. Available: https://www.usenix.org/conference/2002-usenix-annual-technical-conference/my-cache-or-yours-making-storage-more-exclusive

[15] A. R. Karlin, M. S. Manasse, L. Rudolph, and D. D. Sleator, "Competitive snoopy caching," *Algorithmica*, vol. 3, no. 1, pp. 79–119, Nov. 1988. [Online]. Available: https://doi.org/10.1007/BF01762111

[16] K. Gurushankar, N. G. Singer, and B. Subercaseaux, "Latency guarantees for caching with delayed hits," *Proceedings of IEEE INFOCOM*, 2025.

[17] H. Tan, Y. Wang, C. Zhang, G. Li, H. Du, Z. Han, S. H.-C. Jiang, and X.-Y. Li, "Asymptotically tight approximation for online file caching with delayed hits and bypassing," *IEEE Transactions on Networking*, vol. 33, no. 4, pp. 1886–1899, 2025.

[18] D. D. Sleator and R. E. Tarjan, "Amortized efficiency of list update and paging rules," *Commun. ACM*, vol. 28, no. 2, p. 202–208, Feb. 1985. [Online]. Available: https://doi.org/10.1145/2786.2793

[19] A. Fiat, R. M. Karp, M. Luby, L. A. McGeoch, D. D. Sleator, and N. E. Young, "Competitive paging algorithms," *Journal of Algorithms*, vol. 12, no. 4, pp. 685–699, 1991. [Online]. Available: https://www.sciencedirect.com/science/article/pii/019667749190041V

[20] N. Buchbinder and J. S. Naor, "The design of competitive online algorithms via a primal—dual approach," *Foundations and Trends in Theoretical Computer Science*, vol. 3, no. 2-3, pp. 93–263, 05 2009. [Online]. Available: https://doi.org/10.1561/0400000024

[21] A. Adamaszek, A. Czumaj, M. Englert, and H. Räcke, "An o(log k)-competitive algorithm for generalized caching," *ACM Trans. Algorithms*, vol. 15, no. 1, Nov. 2018. [Online]. Available: https://doi.org/10.1145/3280826

[22] S. Chawla, "CS787: advanced algorithms, lecture 14: Online algorithms," Course scribe notes, Dept. Comput. Sci., Univ. Wisconsin–Madison, Madison, WI, USA, 2009, [Online]. Available: https://pages.cs.wisc.edu/~shuchi/courses/787-F09/scribe-notes/lec14.pdf. [Accessed: Aug. 20, 2026].

[23] A. Borodin and R. El-Yaniv, *Online Computation and Competitive Analysis*. Cambridge, U.K.: Cambridge Univ. Press, 1998.

[24] S. O. Krumke, "Online optimization: Competitive analysis and beyond," PhD dissertation, Technische Universität Berlin, Berlin, Germany, 2001.